



**Centro de Investigación e Innovación
en Tecnologías de la Información y
Comunicación — INFOTEC**

Maestría en Ciencia de Datos

El Problema del Transporte: Formulación Primal–Dual, Representación en Grafos y Aplicaciones

Reporte de Investigación — Tema 5: El Problema del Transporte

Asignatura: Matemáticas 2026-1
Imparte: Dra. Briceyda B. Delgado
Equipo: 5
Actividad: 4A — Reporte de Investigación

Integrantes

Andrés Rangel
Santiago Mendoza
David Rodríguez
Bryan Rodríguez

Ciudad de México — 2 de mayo de 2026

El Problema del Transporte: Formulación Primal–Dual, Representación en Grafos y Aplicaciones

A. Rangel, S. Mendoza, D. Rodríguez, B. Rodríguez
Maestría en Ciencia de Datos — INFOTEC, México

Resumen— El Problema del Transporte es uno de los modelos más estudiados de la programación lineal y constituye, junto con su problema dual asociado, un puente natural entre la optimización matemática, la teoría de grafos y la economía de redes. En este reporte se presenta el modelo desde tres ángulos complementarios: (i) la formulación primal del transportista, orientada a minimizar el costo total de distribución sujeto a restricciones de oferta y demanda; (ii) la formulación dual del embarcador, que introduce los precios sombra como herramienta de valoración marginal de los recursos; y (iii) la representación topológica como un grafo bipartito dirigido, donde cada variable de decisión equivale a una arista. Se desarrollan dos ejemplos numéricos resueltos con la biblioteca PuLP de Python: un caso de logística de última milla en e-commerce con datos balanceados, y una distribución de agua en emergencia con restricciones complejas (rutas bloqueadas, capacidades máximas y prioridades obligatorias). El reporte cierra con un análisis comparativo de aplicaciones reales en logística, energía, distribución de emergencia y movilidad urbana, así como con las limitaciones operativas del modelo.

Palabras clave— Programación lineal, problema del transporte, dualidad, precios sombra, grafos bipartitos, PuLP, optimización combinatoria, cadena de suministro.

1. INTRODUCCIÓN

El Problema del Transporte es uno de los modelos clásicos de la programación lineal y, en términos de aplicaciones reales, probablemente el más utilizado. En su forma básica, busca determinar la manera más económica de distribuir un producto homogéneo desde varios puntos de origen donde existe oferta hacia varios puntos de destino donde existe demanda, conocidos los costos unitarios de transporte para cada par origen-destino [1].

El estudio de este modelo es relevante en investigación de operaciones y gestión logística por al menos cinco motivos clave. Primero, su objetivo natural —minimizar el costo total de distribución— afecta directamente el desempeño económico de cualquier empresa que mueva mercancías o recursos. Segundo, aunque su nombre sugiere envíos físicos, su estructura matemática se aplica a problemas que no involucran transporte alguno: programación de la producción donde los periodos actúan como fuentes y destinos, programación de mantenimiento de herramientas, asignación de turnos, y muchos otros casos en los que se requiere mover «algo» (incluso intangible) desde donde sobra hacia donde falta [2].

Tercero, el problema admite algoritmos especializados —como el método de la esquina noroeste, el método del costo mínimo, el método de aproximación de Vogel y el método de los multiplicadores— que son sustancialmente más eficientes que el simplex general para esta estructura particular. Cuarto, el modelo es uno de los pilares de la cadena de suministro moderna: gestionar el flujo de bienes, dinero e información entre proveedores, fabricantes y consumidores requiere un equilibrio constante entre capacidad de respuesta y eficiencia, y el problema del transporte es la herramienta básica para encontrar ese equilibrio. Quinto, su representación visual mediante redes (nodos y arcos) y mediante el tablero de transporte facilita la comunicación entre analistas y tomadores de decisiones [2].

Adicionalmente, el problema admite una *interpretación dual* sumamente útil. La teoría de la dualidad en programación lineal establece que todo programa lineal —el primal— tiene asociado

otro programa lineal —el dual—, y que ambos comparten el mismo valor óptimo. En el caso del transporte, mientras el primal representa al transportista que minimiza costos, el dual representa al embarcador que compra producto en los orígenes y lo vende en los destinos buscando maximizar el margen [3]. Las variables del dual, conocidas como **precios sombra**, miden el cambio marginal del costo óptimo ante variaciones unitarias en la oferta o la demanda, y son la base teórica del método de los multiplicadores que prueba la optimalidad de una solución [2].

El presente reporte sigue una estructura clara: la Sección 2 ofrece un estado del arte que recorre los hitos históricos del problema desde Monge hasta los algoritmos modernos; la Sección 3 desarrolla la formulación matemática con los enfoques primal, dual y de grafos; la Sección 4 resuelve dos ejemplos numéricos completos con código en Python; la Sección 5 expone aplicaciones reales del modelo y sus limitaciones; la Sección 6 presenta las conclusiones del equipo; finalmente, la Sección 7 lista las referencias bibliográficas consultadas.

2. ESTADO DEL ARTE

Esta sección recorre la evolución del Problema del Transporte desde sus orígenes geométricos en el siglo XVIII hasta su consolidación algorítmica en la segunda mitad del siglo XX. La **Figura 1** resume gráficamente los hitos principales.

2.1. Modelo geométrico de Monge (1781)

El trabajo más antiguo conocido sobre el problema se debe a Gaspard Monge, matemático francés del siglo XVIII. En su *Mémoire sur la théorie des déblais et des remblais* (1781), publicado en las memorias de la Academia Francesa de Ciencias, Monge planteó el problema de mover un volumen de tierra desde varios puntos hacia varios destinos minimizando el trabajo total. Su interés provenía de problemas de ingeniería militar relacionados con la construcción de fortificaciones [1]. Aunque su lenguaje era geométrico —hablaba de partículas infinitamente pequeñas que se corresponden uno a uno—, su intuición de costos era ya muy similar al actual costeo por tonelada-kilómetro:

Hitos históricos del Problema del Transporte (1781-1956)

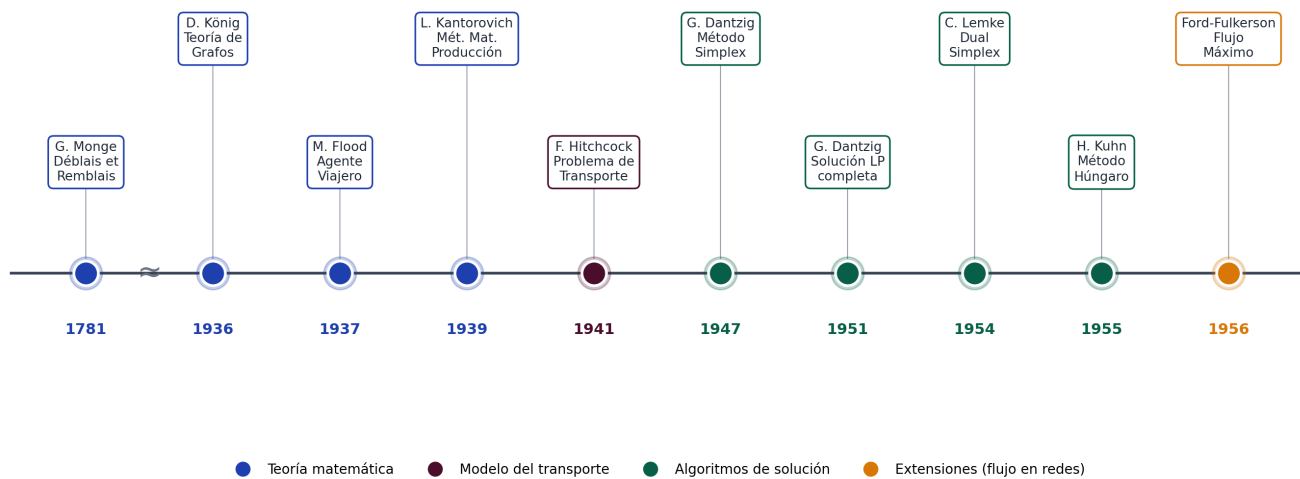


Figura 1. Hitos históricos del Problema del Transporte. La cronología se divide en tres eras: *Pre-IO* (siglo XVIII, fundamentos geométricos de Monge), *Fundamentos teóricos* (1936–1941, formalización del problema por König, Flood, Kantorovich y Hitchcock) y *Algoritmos y computación* (1947–1956, era del simplex y los flujos en redes). Elaboración propia con base en [1] y [2].

el precio de mover una partícula es proporcional a su peso y a la distancia recorrida.

2.2. Teoría de grafos de König (1936)

Dénes König publicó en 1936 *Theorie der endlichen und unendlichen Graphen*, considerada la primera obra de texto formal dedicada a la teoría de grafos. Su trabajo proporcionó el primer tratamiento sistemático de la teoría de grafos, formalizando conceptos como conectividad, ciclos y caminos que más tarde resultarían esenciales para representar problemas de flujo en redes. En particular, el llamado **lema de König**—que garantiza la existencia de un camino infinito en cualquier árbol infinito con un número finito de aristas por nodo— es un resultado seminal en la teoría de grafos infinita.

2.3. Problema del agente viajero de Flood (1937)

El matemático estadounidense Merrill M. Flood fue uno de los principales impulsores del **Problema del Agente Viajero** (TSP) en la década de 1930, contribuyendo a su popularización en la comunidad de investigación de operaciones. El TSP consiste en encontrar la ruta más corta para visitar un conjunto de ciudades una sola vez y regresar al origen. Aunque el TSP es un problema de optimización combinatoria fundamentalmente distinto al problema de transporte—este último tiene una estructura de restricciones totalmente unimodular que lo hace “fácil” (resoluble en tiempo polinomial), mientras que el TSP es *NP-hard*—, ambos comparten el concepto de minimizar un costo asociado a una red y el trabajo de Flood motivó el desarrollo de métodos de solución para problemas de asignación. Flood llegó al problema desde una aplicación concreta—el enrutamiento de autobuses escolares en Nueva Jersey— y junto con Dresher y Robinson en RAND Corporation impulsó su estudio formal en los años cincuenta.

2.4. Métodos matemáticos de Kantorovich (1939)

El matemático ruso Leonid V. Kantorovich propuso en su obra *Métodos matemáticos para la organización y la producción* (1939) la primera formulación de un problema de asignación productiva como un programa lineal. Su enfoque buscaba el uso más eficiente de materiales y mano de obra mediante «valoraciones determinadas objetivamente», antecedentes directos de los precios sombra. Por este aporte recibió el Premio Nobel de Economía en 1975, compartido con T. C. Koopmans.

2.5. Modelo de Hitchcock (1941)

Frank L. Hitchcock publicó en 1941 el artículo «The distribution of a product from several sources to numerous localities», donde formuló de manera explícita el problema de minimizar el costo de distribuir un producto desde varias fábricas hacia un número dado de ciudades. Por su claridad y generalidad, el modelo se conoce desde entonces como el **Problema de Transporte de Hitchcock**. Trabajos posteriores y de manera independiente fueron desarrollados por Kantorovich (1942) y Koopmans (1947) [1].

2.6. Programación lineal y el método simplex (1947)

George B. Dantzig formalizó en 1947 la programación lineal como disciplina autónoma y desarrolló el **método simplex**, un algoritmo iterativo que permite resolver problemas con cualquier número de variables. El simplex fue elegido como uno de los diez algoritmos más importantes del siglo XX y sigue siendo, junto con sus variantes, el caballo de batalla de la programación lineal. En 1951 Dantzig adaptó el método para resolver específicamente el problema del transporte [1].

2.7. Resolución computacional (década de 1950)

La primera implementación significativa del problema en una computadora se gestó a inicios de los años cincuenta. En 1951,

A. Orden dirigió un esfuerzo en la computadora SEAC para resolver un modelo de despliegue de la Fuerza Aérea estadounidense, uno de los primeros casos documentados de aplicación de la investigación de operaciones en hardware real. Poco después, en RAND Corporation, Dantzig y Orchard-Hays desarrollaron los primeros códigos especializados, y más tarde aparecieron el método de la esquina noroeste, el método del costo mínimo y el método de aproximación de Vogel.

2.8. Método dual simplex (1954) y método húngaro (1955)

Carlton E. Lemke desarrolló en 1954 el **método dual simplex**, particularmente útil cuando se añaden nuevas restricciones a un problema ya resuelto, pues parte de una solución básica óptima pero infactible y avanza hacia la factibilidad. Un año después, Harold W. Kuhn formalizó el **método húngaro**, un algoritmo polinomial específico para el problema de asignación —caso particular del transporte— basado en la reducción sistemática de la matriz de costos.

2.9. Algoritmos de flujo en redes (Ford–Fulkerson, 1956)

L. R. Ford y D. R. Fulkerson popularizaron en 1956 su algoritmo de flujo máximo, que generalizó el problema del transporte al caso de redes con capacidades en los arcos. Su trabajo abrió la puerta a una familia entera de problemas de optimización combinatoria sobre redes (transbordo, asignación, flujo de costo mínimo) que extienden el modelo clásico.

2.10. Desarrollos contemporáneos

En las últimas décadas, la investigación se ha movido hacia variantes que incorporan incertidumbre (programación estocástica), múltiples objetivos (programación multicriterio), múltiples actores con intereses contrapuestos (programación bi-nivel y teoría de juegos) y restricciones combinatorias (programación entera mixta). El modelo gravitacional de distribución de viajes [4] y los modelos de equilibrio espacial de precios [5] son ejemplos relevantes de extensiones que conservan la estructura matemática esencial del transporte pero añaden realismo.

3. DESARROLLO Y METODOLOGÍA

Esta sección desarrolla el modelo en tres capas: la formulación primal del transportista (3.1), el problema dual del embarcador con los precios sombra (3.2), y la representación topológica como grafo bipartito (3.3).

3.1. Formulación primal: el enfoque del transportista

Considérese una red con m orígenes y n destinos. Cada origen i dispone de una oferta a_i y cada destino j requiere una demanda b_j . El costo unitario de enviar producto desde el origen i al destino j se denota c_{ij} , y la variable de decisión x_{ij} representa la cantidad efectivamente enviada por esa ruta. Se asume el caso balanceado, $\sum a_i = \sum b_j$, en cuyo caso las restricciones se escriben como igualdades [2]. Nótese que en el caso balanceado, las restricciones de desigualdad $\sum_j x_{ij} \leq a_i$ y $\sum_i x_{ij} \geq b_j$ son equivalentes a las igualdades (2) y (3), ya que la suma de todas las ofertas es igual a la suma de todas las demandas. Cualquier solución factible que no cumpla una igualdad implicaría un desbalance imposible en el sistema cerrado. El programa lineal asociado es:

$$\text{Min } Z = \sum_{i=1}^m \sum_{j=1}^n c_{ij} x_{ij} \quad (1)$$

$$\text{s. a. } \sum_{j=1}^n x_{ij} = a_i, \quad i = 1, \dots, m \quad (2)$$

$$\sum_{i=1}^m x_{ij} = b_j, \quad j = 1, \dots, n \quad (3)$$

$$x_{ij} \geq 0, \quad \forall i, j \quad (4)$$

La función objetivo (1) refleja el interés del transportista: minimizar el costo total del plan de distribución. Las restricciones (2) y (3) garantizan, respectivamente, que cada origen agote exactamente su oferta y que cada destino reciba exactamente su demanda. La restricción (4) impide envíos negativos. En forma matricial compacta, el problema se escribe como

$$\text{mín } \mathbf{c}^T \mathbf{x} \quad \text{s. a. } \mathbf{Ax} = \mathbf{b}, \quad \mathbf{x} \geq \mathbf{0}$$

donde $\mathbf{x}$ es el vector que reúne las $m \cdot n$ variables x_{ij} , $\mathbf{c}$ el vector de costos unitarios, $\mathbf{b}$ el vector que concatena ofertas y demandas, y $\mathbf{A}$ la matriz de incidencia de tamaño $(m+n) \times (m \cdot n)$, en la cual cada columna —correspondiente a la variable x_{ij} — contiene exactamente dos entradas iguales a 1: una en la fila de la restricción de oferta del origen i y otra en la fila de la restricción de demanda del destino j . Esta estructura de red le confiere la propiedad de ser **totalmente unimodular**, lo que garantiza que, si los datos son enteros, la solución básica óptima también será entera.

El modelo es **balanceado** cuando la oferta total iguala a la demanda total. Cuando no lo es —cosa que ocurre con frecuencia en la práctica—, se recurre a dos ajustes estándar:

- Si la oferta excede la demanda, se introduce un **destino ficticio** con costos unitarios cero que absorbe el excedente sin penalizar la función objetivo.
- Si la demanda excede la oferta, se introduce un **origen ficticio** cuya cantidad enviada representa la demanda no satisfecha, a la que normalmente se le asigna un costo de penalización elevado.

3.1.1. Componentes esenciales del modelo

Los elementos que conviene tener presentes para reproducir cualquier instancia son: las *fuentes* y *destinos* (el grafo bipartito subyacente), los *parámetros* (oferta a_i , demanda b_j y costos c_{ij}), las *variables de decisión* x_{ij} y, eventualmente, los nodos ficticios para balancear el modelo. El número total de variables del problema crece como $m \cdot n$, mientras que el número de restricciones es $m + n$ (más la no negatividad).

3.1.2. Procedimiento general de solución

Cuando se resuelve a mano, el procedimiento clásico consta de tres pasos: (i) obtener una solución básica factible inicial mediante la esquina noroeste, el costo mínimo o Vogel; (ii) probar optimalidad mediante el método de los multiplicadores (u - v), que asocia variables a las restricciones de fila y columna y exige

$u_i + v_j = c_{ij}$ para las celdas con flujo; (iii) si la solución no es óptima, mejorarla construyendo un lazo cerrado para reasignar unidades hacia rutas más baratas. En la práctica moderna, sin embargo, estos pasos los realiza un solver de programación lineal en milisegundos, como se muestra en los ejemplos de la Sección 4.

3.2. Teoría de dualidad y precios sombra

A cada restricción del primal le corresponde una variable del dual. Asociando a las restricciones de oferta (2) las variables u_i y a las de demanda (3) las variables v_j , el dual del problema (1)–(4) resulta [3]:

$$\text{Max } W = \sum_i a_i u_i + \sum_j b_j v_j \quad (5)$$

$$\text{s. a. } u_i + v_j \leq c_{ij}, \quad \forall i, j \quad (6)$$

$$u_i, v_j \text{ irrestrictas en signo} \quad (7)$$

En forma matricial compacta, el dual se escribe como máx $\mathbf{b}^\top \mathbf{y}$ sujeto a $\mathbf{A}^\top \mathbf{y} \leq \mathbf{c}$, donde $\mathbf{y} = (u_1, \dots, u_m, v_1, \dots, v_n)$ y $\mathbf{A}$ es la misma matriz de incidencia del primal.

3.2.1. Interpretación económica

Mientras el primal representa al *transportista* que minimiza costos, el dual representa al *embarcador* que compra el producto en los orígenes y lo vende en los destinos buscando maximizar la utilidad de la operación [3]. Las variables duales adquieren entonces un significado económico muy concreto:

- u_i — **precio sombra en el origen**: valor marginal de contar con una unidad adicional de oferta en el origen i . Mide el impacto en el costo óptimo total: $\partial Z^* / \partial a_i = u_i^*$.
- v_j — **precio sombra en el destino**: costo marginal de satisfacer una unidad adicional de demanda en el destino j . Se cumple $\partial Z^* / \partial b_j = v_j^*$.

La restricción (6), $u_i + v_j \leq c_{ij}$, expresa una condición de equilibrio espacial de precios: el valor marginal de enviar una unidad del origen i al destino j no puede exceder el costo de hacerlo. Si $u_i + v_j < c_{ij}$, resulta más costoso mover el producto que el valor que genera, por lo que la ruta permanece inactiva. Cournot ya había formalizado este principio de equilibrio espacial en 1838: «un artículo capaz de ser transportado debe fluir del mercado donde su valor es menor al mercado donde su valor es más grande, hasta que la diferencia en valor, de un mercado al otro, no represente más que el costo del transporte» [5].

3.2.2. Holgura complementaria

La teoría de dualidad establece que en la solución óptima se cumple para cada par (i, j) :

$$x_{ij}^* \cdot (c_{ij} - u_i^* - v_j^*) = 0 \quad (8)$$

De esta ecuación se desprenden dos casos lógicos. Primero, si la ruta es **activa** ($x_{ij}^* > 0$), entonces $u_i^* + v_j^* = c_{ij}$: el precio sombra en el destino es exactamente el precio sombra en el origen más el costo de transporte. Esto constituye un equilibrio

de mercado perfecto. Segundo, si la ruta es **deficitaria** ($u_i^* + v_j^* < c_{ij}$), entonces necesariamente $x_{ij}^* = 0$: trasladar producto por esa ruta destruiría valor, así que el flujo se anula. La **Figura 2** resume gráficamente la dualidad primal-dual.

3.3. Representación mediante grafos bipartitos

El problema admite una representación natural como un **grafo bipartito dirigido** $G = (V, E)$ donde el conjunto de vértices se parte en dos subconjuntos disjuntos: orígenes S y destinos D . Cada origen i se asocia a una divergencia de flujo positiva ($+a_i$), y cada destino j a una divergencia negativa ($-b_j$). Las aristas conectan únicamente nodos de S con nodos de D ; no existen aristas entre dos orígenes ni entre dos destinos. La **Figura 3** ilustra esta estructura para un caso pequeño con dos orígenes y tres destinos.

Existe una correspondencia exacta entre la estructura del grafo y la estructura algebraica del problema: **el número de aristas del grafo bipartito completo es igual al número de variables de decisión** del programa lineal, es decir, $m \cdot n$. Cada variable x_{ij} representa el flujo que circula por la arista (i, j) .

Una propiedad fundamental es que las restricciones de oferta y demanda en el caso balanceado contienen una ecuación redundante (la suma de las ofertas iguala a la suma de las demandas), por lo que el rango efectivo del sistema es $m + n - 1$. Esto implica que la solución óptima activará a lo sumo $m + n - 1$ aristas, las cuales formarán topológicamente un **árbol de expansión**: una estructura sin ciclos que conecta todos los nodos relevantes. Esta característica garantiza que no haya rutas redundantes en el plan óptimo. Adicionalmente, la matriz de coeficientes asociada a este tipo de redes es *totalmente unimodular*, lo que asegura que la solución óptima de la relajación lineal sea entera siempre que la oferta y la demanda lo sean.

3.3.1. Implementación computacional de referencia

A continuación se presenta una implementación genérica que materializa los tres elementos teóricos discutidos hasta aquí: la formulación primal (1)–(4), la extracción de los precios sombra del dual (Sección 3.2) y la visualización topológica como grafo bipartito. El código emplea PuLP para la optimización y NetworkX para el dibujo del árbol de expansión óptimo. Sirve como plantilla que los ejemplos de la Sección 4 particularizan con datos reales:

```
"""
Análisis Computacional del Problema de
Transporte Bipartito.
Optimización Primal-Dual (PuLP), Extracción
de Precios Sombra y Topología (NetworkX).
"""

import pulp
import networkx as nx
import matplotlib.pyplot as plt

def instanciar_y_resolver_transporte():
    # 1. Definición de Parámetros
    origenes = ["Planta_A", "Planta_B", "Planta_C"]
    oferta = {"Planta_A": 2000, "Planta_B": 1500,
              "Planta_C": 1000}

    destinos = ["Mercado_1", "Mercado_2",
                "Mercado_3", "Mercado_4"]
    demanda = {"Mercado_1": 1300, "Mercado_2": 1800,
               "Mercado_3": 900, "Mercado_4": 500}
```

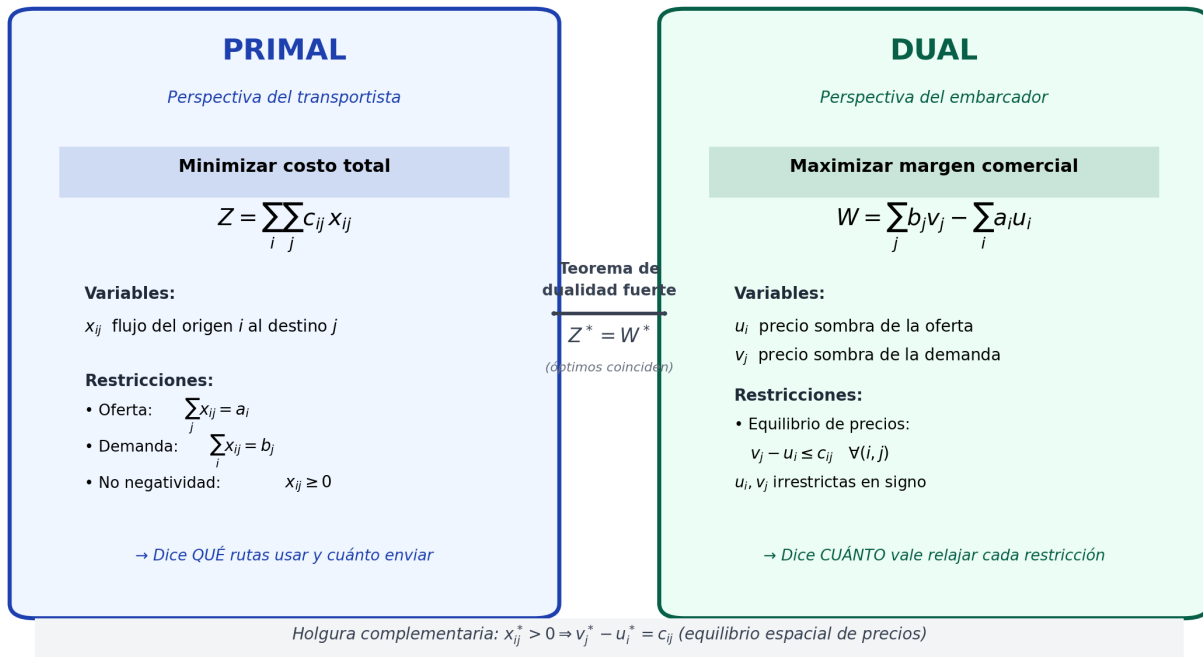



Figura 2. Esquema integral de la relación primal–dual. El problema primal busca minimizar el costo total desde la perspectiva del transportista, mientras que su dual busca maximizar la suma de los precios sombra ponderados por oferta y demanda. La teoría de dualidad fuerte garantiza que ambos programas alcanzan el mismo valor en el óptimo ($Z^* = W^*$). La condición de holgura complementaria, mostrada en la banda inferior, conecta las soluciones primal y dual: cuando una ruta tiene flujo positivo, el precio sombra en el destino iguala al del origen más el costo de transporte, reproduciendo el principio de equilibrio espacial.

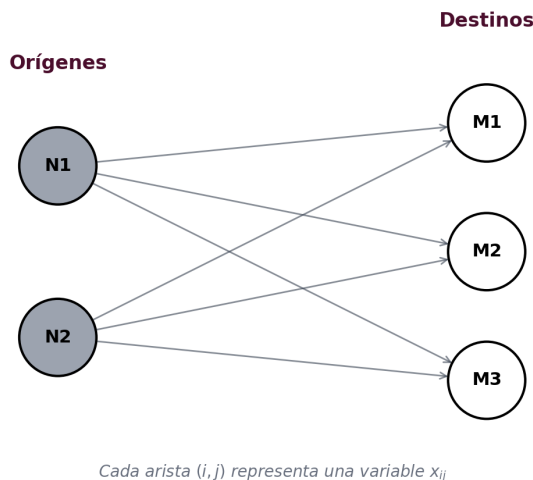


Figura 3. Grafo bipartito genérico de un problema de transporte con dos orígenes (N_1, N_2) y tres destinos (M_1, M_2, M_3). Las aristas dirigidas $(i, j) \in E$ con costo c_{ij} conectan exclusivamente nodos de S con nodos de D ; el número total de aristas $|S| \times |D|$ coincide con el número de variables de decisión x_{ij} del programa lineal. Adaptado de [3].

```

costos = {
    "Planta_A": {"Mercado_1":10, "Mercado_2":15,
                 "Mercado_3":20, "Mercado_4":12},
    "Planta_B": {"Mercado_1":12, "Mercado_2":10,
                 "Mercado_3":15, "Mercado_4":18},
    "Planta_C": {"Mercado_1":15, "Mercado_2":25,
                 "Mercado_3":10, "Mercado_4":14},
}

# 2. Inicializacion del Modelo Primal
modelo = pulp.LpProblem("Modelo_Transporte",
                        pulp.LpMinimize)

# 3. Aristas equivalentes (variables x_ij)
rutas = [(i, j) for i in origenes
           for j in destinos]
flujos = pulp.LpVariable.dicts(
    "Carga", rutas, lowBound=0,
    cat=pulp.LpContinuous)

# 4. Funcion objetivo Z
modelo += pulp.lpSum(flujos[(i, j)]*costos[i][j]
                     for (i, j) in rutas)

# 5. Balance de masa (oferta y demanda)
for i in origenes:
    modelo += (pulp.lpSum(flujos[(i, j)]
                          for j in destinos) == oferta[i],
               f"Oferta_{i}")
for j in destinos:
    modelo += (pulp.lpSum(flujos[(i, j)]
                          for i in origenes) == demanda[j],
               f"Demanda_{j}")
  
```

```

# 6. Resolución
modelo.solve(pulp.PULP_CBC_CMD(msg=False))
print(f"Z* = ${pulp.value(modelo.objective):.2f}")

# 7. Aristas activas y precios sombra
activas = []
for (i, j) in rutas:
    v = flujos[(i, j)].varValue
    if v and v > 0:
        print(f"{i} -> {j}: {v} unidades")
        activas.append((i, j, v))

print("--- Precios sombra ---")
for i in origenes:
    u_i = modelo.constraints[f"Oferta_{i}"].pi
    print(f"u_{i} = {u_i}")
for j in destinos:
    v_j = modelo.constraints[f"Demanda_{j}"].pi
    print(f"v_{j} = {v_j}")

# 8. Grafo bipartito del arbol optimo
generar_grafo_bipartito(origenes, destinos,
                        activas)

def generar_grafo_bipartito(origenes, destinos,
                            aristas):
    """Visualiza el arbol de expansion optimo."""
    G = nx.DiGraph()
    G.add_nodes_from(origenes, bipartite=0)
    G.add_nodes_from(destinos, bipartite=1)
    for (u, v, flujo) in aristas:
        G.add_edge(u, v, weight=flujo)

    plt.figure(figsize=(10, 6))
    pos = nx.bipartite_layout(G, origenes)
    nx.draw_networkx_nodes(G, pos, nodelist=origenes,
                          node_color='skyblue', node_size=2000)
    nx.draw_networkx_nodes(G, pos, nodelist=destinos,
                          node_color='lightgreen', node_size=2000)
    nx.draw_networkx_edges(G, pos, arrowstyle='->',
                          edge_color='gray', width=2)
    nx.draw_networkx_labels(G, pos, font_size=10,
                          font_weight='bold')

    etiquetas = {(u, v): f"{flujo}"
                  for u, v, flujo in aristas}
    nx.draw_networkx_edge_labels(G, pos,
                                edge_labels=etiquetas, font_size=9)

    plt.title("Grafo Bipartito: Arbol de "
              "Expansion Optimo", fontsize=14)
    plt.axis('off')
    plt.show()

if __name__ == "__main__":
    instanciar_y_resolver_transporte()

```

Esta implementación de referencia es deliberadamente genérica: cualquier instancia concreta del problema se obtiene reemplazando los diccionarios de oferta, demanda y costos. Los dos ejemplos de la Sección 4 siguen exactamente este patrón, agregando en el segundo caso restricciones de capacidad, bloqueos y prioridades sobre las variables.

4. EJEMPLOS PRÁCTICOS RESUELTOS

Esta sección presenta dos ejemplos numéricos del Problema del Transporte resueltos íntegramente con la biblioteca **PuLP** de Python. El Ejemplo 1 (Sección 4.1) modela una red de logística de última milla en un escenario de e-commerce balanceado. El Ejemplo 2 (Sección 4.2) extiende el modelo anterior con res-

tricciones adicionales que reflejan situaciones operativas reales: una ruta físicamente bloqueada, capacidades máximas en ciertas tuberías y una condición de prioridad obligatoria. Ambos ejemplos permiten apreciar la flexibilidad del modelo y la utilidad práctica del análisis de los precios sombra.

4.1. Ejemplo 1: Logística de última milla en e-commerce

4.1.1. Caso de estudio

Durante la temporada comercial del «Buen Fin 2026», la empresa Mercado Libre (en adelante, «la empresa») proyecta un pico histórico de ventas para el lanzamiento del iPhone 18. Para garantizar el cumplimiento de las entregas bajo la modalidad «Full» (entrega al día siguiente), la compañía requiere preposicionar su inventario de manera estratégica antes de que comience el evento.

La empresa cuenta con tres Mega Centros de Distribución (orígenes) que en conjunto tienen almacenados 120 lotes del producto, donde un lote equivale a 100 unidades. Estos lotes deben distribuirse a cinco Centros de Última Milla (destinos), cuya demanda proyectada suma exactamente 120 lotes. Como la oferta total iguala a la demanda total, el sistema se clasifica como un **problema de transporte balanceado** [2]. Las **Tablas I y II** resumen oferta y demanda; la **Tabla III** presenta la matriz de costos de transporte terrestre, expresados en pesos mexicanos por lote.

Tabla 1. Oferta disponible por Mega Centro (orígenes).

Origen i	Ubicación	Lotes (a_i)
1	Cuautitlán Izcalli	40
2	Monterrey	35
3	Guadalajara	45
Total		120

Tabla 2. Demanda requerida por Centro de Última Milla (destinos).

Destino j	Ubicación	Lotes (b_j)
1	Tijuana	20
2	Puebla	25
3	Querétaro	30
4	Mérida	15
5	Hermosillo	30
Total		120

Tabla 3. Matriz de costos c_{ij} en MXN por lote.

Origen	Tij.	Pue.	Qro.	Mér.	Hmo.
Cuautitlán	2,800	500	700	3,200	2,500
Monterrey	2,400	1,800	1,200	3,800	1,600
Guadalajara	2,200	1,100	600	3,500	1,900

4.1.2. Representación en grafo bipartito

El sistema se modela mediante un grafo bipartito dirigido con tres nodos de origen y cinco de destino, lo que da un total de **15 aristas** posibles —es decir, 15 variables de decisión x_{ij} . La **Figura 4** muestra todas las rutas potenciales antes de que el modelo decida cuáles activar. Por la regla de redes mencionada en la Sección 3.3, la solución óptima activará a lo sumo $m + n - 1 = 3 + 5 - 1 = 7$ rutas; en este caso particular, el algoritmo encontrará el óptimo con sólo 6 rutas activas.

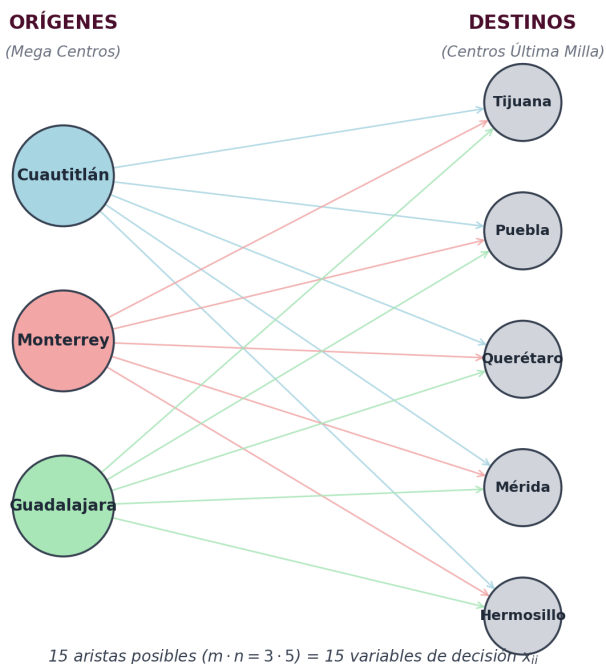


Figura 4. Grafo bipartito completo del Ejemplo 1: tres orígenes (Cuautitlán, Monterrey, Guadalajara, identificados por color) conectados con cinco destinos (en gris) mediante 15 aristas posibles. Cada arista representa una variable de decisión x_{ij} . La teoría de redes garantiza que la solución óptima activará a lo sumo $m + n - 1 = 7$ aristas, formando un árbol de expansión.

4.1.3. Formulación matemática

Sea x_{ij} la cantidad de lotes enviados desde el origen i al destino j , con $i \in \{1, 2, 3\}$ y $j \in \{1, \dots, 5\}$. La función objetivo, particularizando la fórmula (1) con los costos de la Tabla III, queda:

$$\begin{aligned} \text{Min } Z = & 2800x_{11} + 500x_{12} + 700x_{13} + \\ & 3200x_{14} + 2500x_{15} + 2400x_{21} + \\ & 1800x_{22} + 1200x_{23} + 3800x_{24} + \\ & 1600x_{25} + 2200x_{31} + 1100x_{32} + \\ & 600x_{33} + 3500x_{34} + 1900x_{35} \end{aligned}$$

Con las cinco restricciones de oferta y demanda balanceadas (omitidas aquí por brevedad), y la condición de no negatividad $x_{ij} \geq 0$ para todo (i, j) .

4.1.4. Resolución computacional

Por la dimensionalidad del modelo (15 variables, 8 restricciones de igualdad), la resolución manual sería tediosa pero perfectamente factible. En la práctica se utiliza la biblioteca **PuLP**, un modelador de programación lineal en Python que se conecta automáticamente con solvers como CBC o GLPK. PuLP permite declarar la función objetivo y las restricciones con una sintaxis natural y devuelve, además del plan óptimo, los precios sombra como subproducto del solver. El siguiente fragmento contiene el código completo:

```
import pulp

origenes = ['Cuautitlan', 'Monterrey',
            'Guadalajara']
destinos = ['Tijuana', 'Puebla', 'Queretaro',
            'Merida', 'Hermosillo']
oferta = {'Cuautitlan': 40, 'Monterrey': 35,
          'Guadalajara': 45}
demanda = {'Tijuana': 20, 'Puebla': 25,
           'Queretaro': 30, 'Merida': 15,
           'Hermosillo': 30}

costos = {
    'Cuautitlan': {'Tijuana': 2800, 'Puebla': 500,
                  'Queretaro': 700, 'Merida': 3200,
                  'Hermosillo': 2500},
    'Monterrey': {'Tijuana': 2400, 'Puebla': 1800,
                  'Queretaro': 1200, 'Merida': 3800,
                  'Hermosillo': 1600},
    'Guadalajara': {'Tijuana': 2200, 'Puebla': 1100,
                   'Queretaro': 600, 'Merida': 3500,
                   'Hermosillo': 1900},
}

prob = pulp.LpProblem("Logistica",
                      pulp.LpMinimize)
rutas = [(o, d) for o in origenes
          for d in destinos]
x = pulp.LpVariable.dicts("Ruta",
                          (origenes, destinos),
                          lowBound=0)

# Funcion objetivo
prob += pulp.lpSum(x[o][d] * costos[o][d]
                  for (o, d) in rutas)

# Restricciones de oferta
for o in origenes:
    prob += pulp.lpSum(x[o][d]
                      for d in destinos) == oferta[o]

# Restricciones de demanda
for d in destinos:
    prob += pulp.lpSum(x[o][d]
                      for o in origenes) == demanda[d]

prob.solve(pulp.PULP_CBC_CMD(msg=False))
print(f"Z* = ${pulp.value(prob.objective):.0f}")

for v in prob.variables():
    if v.varValue > 0:
        print(f"{v.name}: {int(v.varValue)} lotes")
```

Al ejecutar el script, el solver CBC encuentra la solución óptima global en aproximadamente 0.05 segundos. La salida del programa es:

```
Z* = $171,500
Ruta_Cuautitlan_Merida: 15 lotes
Ruta_Cuautitlan_Puebla: 25 lotes
Ruta_Guadalajara_Queretaro: 30 lotes
```


Plan óptimo de distribución

Ejemplo 1: Logística de última milla — Buen Fin 2026

120 lotes balanceados · $Z^* = \$171,500$ MXN

Origen / Destino	Tijuana (20)	Puebla (25)	Querétaro (30)	Mérida (15)	Hermosillo (30)
Cuautitlán (40)	\$2,800	25 lotes \$500/lote = \$12,500	\$700	15 lotes \$3,200/lote = \$48,000	\$2,500
Monterrey (35)	5 lotes \$2,400/lote = \$12,000	\$1,800	\$1,200	\$3,800	30 lotes \$1,600/lote = \$48,000
Guadalajara (45)	15 lotes \$2,200/lote = \$33,000	\$1,100	30 lotes \$600/lote = \$18,000	\$3,500	\$1,900

Resultados clave

- 6 rutas activas (de 15 posibles)
- 9 rutas descartadas: el modelo las identifica como ineficientes
- Forma topológica: árbol de expansión (sin ciclos)
- Cumple la propiedad teórica: $m + n - 1 = 7$ aristas máximo

Hallazgo gerencial

Monterrey está geográficamente más cerca de Tijuana, pero el plan global asigna Monterrey → Hermosillo.

→ Minimizar el costo de una ruta aislada no garantiza minimizar el costo de la red completa.

Figura 5. Plan óptimo de distribución del Ejemplo 1. Las celdas en rojo intenso corresponden a flujos altos; las celdas grises representan rutas no utilizadas. Los costos unitarios están expresados en MXN por lote. El subtotal de cada ruta activa aparece debajo del flujo en color rojo oscuro.

```
Ruta_Guadalajara_Tijuana: 15 lotes
Ruta_Monterrey_Hermosillo: 30 lotes
Ruta_Monterrey_Tijuana: 5 lotes
```

4.1.5. Resultados e interpretación

El costo mínimo del plan logístico es $Z^* = \$171,500$ MXN. La **Figura 5** muestra la matriz de asignación óptima: las celdas resaltadas corresponden a las rutas activas, con el flujo asignado, el costo unitario y el subtotal de cada ruta. La **Figura 6** compara este plan óptimo con un plan “intuitivo” que un planeador humano podría haber diseñado siguiendo la lógica geográfica de proximidad, evidenciando un ahorro de \$7,000 (3.9 %) gracias a la optimización global. De las 15 rutas posibles del planteamiento inicial, el modelo descartó 9 (asignándoles flujo cero), dejando sólo las 6 que minimizan el costo total. La **Figura 7** visualiza estas 6 rutas como un árbol de expansión, materializando la propiedad teórica de que la solución óptima activa a lo sumo $m + n - 1 = 7$ aristas.

Análisis de las decisiones del modelo. Los resultados ilustran cómo la optimización basada en datos puede contradecir la intuición geográfica. Monterrey está considerablemente más cerca de Tijuana que Guadalajara, y un planeador podría haber asignado intuitivamente todo el inventario de Monterrey para cubrir Tijuana. Sin embargo, el modelo global determina que la capacidad de Monterrey genera mayor rentabilidad si se destina

a Hermosillo, dejando que Guadalajara —con un costo unitario más bajo hacia Tijuana— cubra el faltante. La conclusión gerencial es directa: minimizar el costo de una ruta aislada no garantiza minimizar el costo total de la red.

Consideraciones operativas adicionales

El modelo entrega la solución de menor costo, pero la implementación real durante una temporada de alta demanda como el «Buen Fin» requiere evaluar también restricciones de tiempo. Por ejemplo, la ruta activa Cuautitlán → Mérida (aproximadamente 1,300 km) cumple con el criterio de costo mínimo terrestre, pero excede operativamente el umbral de 24 horas necesario para cumplir la promesa de entrega «Full» al día siguiente. Para iteraciones futuras del modelo se recomienda penalizar ese tipo de rutas largas mediante la técnica de la «M grande»: asignar un costo artificialmente alto a las rutas que excedan el tiempo de tránsito permitido obliga al algoritmo a evitarlas salvo que sea estrictamente indispensable usarlas.

4.1.6. Análisis dual: precios sombra

Resolviendo el modelo con variables continuas (en lugar de enteras) para que el solver reporte las variables duales, se obtienen los precios sombra de la **Tabla IV**. Cada precio sombra responde la pregunta: ¿Qué tan costoso es abastecer a este nodo en particular en comparación con el resto del sistema logístico?.

Comparativo: decisión intuitiva vs plan óptimo del modelo

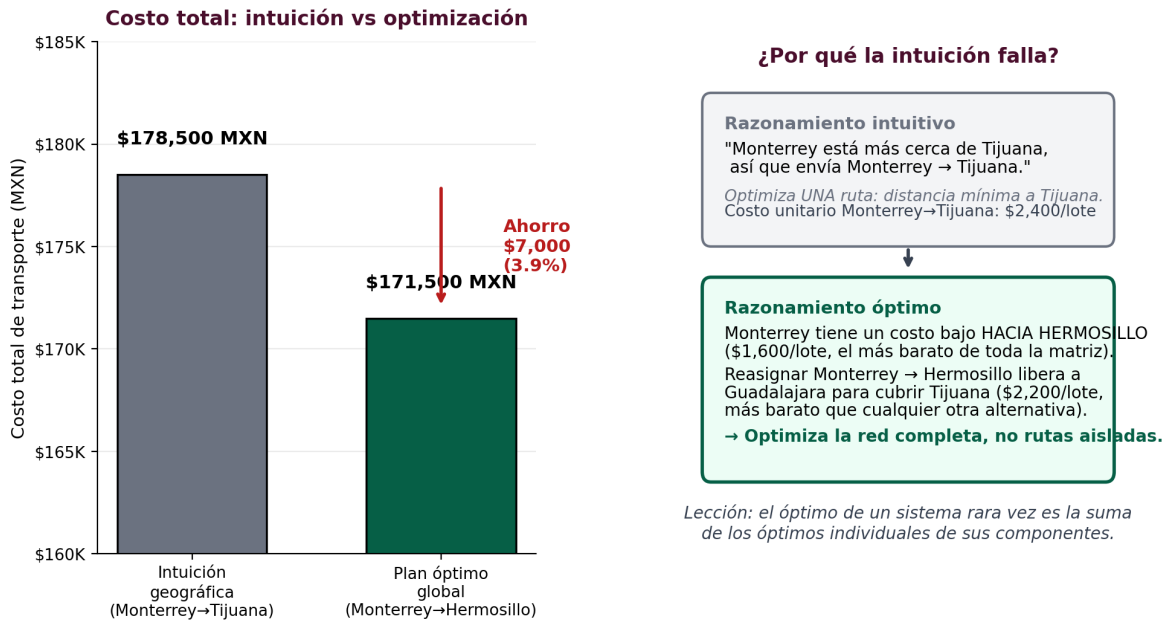


Figura 6. Comparativo *intuición vs. optimización* para el Ejemplo 1. **Izquierda:** el plan “intuitivo” que asigna todo Tijuana a Monterrey (la ciudad geográficamente más cercana) cuesta \$178,500 MXN, mientras que el plan óptimo del modelo cuesta \$171,500, un ahorro de \$7,000 (3.9 %).

Derecha: explicación del razonamiento detrás de cada estrategia. El modelo redirige inventario para evitar el costo unitario más alto de Monterrey → Tijuana (\$2,400/lote) en favor de Guadalajara → Tijuana (\$2,200/lote), liberando capacidad de Monterrey hacia Hermosillo. La conclusión gerencial es directa: minimizar el costo de una ruta aislada no garantiza minimizar el costo total de la red.

Tabla 4. Precios sombra del Ejemplo 1.

Restricción	π (MXN)
Oferta Cuautitlán	500
Oferta Monterrey	1,000
Oferta Guadalajara	800
Demanda Tijuana	1,400
Demanda Puebla	0
Demanda Querétaro	-200
Demanda Mérida	2,700
Demanda Hermosillo	600

Los valores reportados son los multiplicadores de las restricciones en el óptimo, obtenidos numéricamente por el solver CBC con una precisión de 10^{-6} . Los signos pueden interpretarse directamente como los precios sombra duales u_i (oferta) y v_j (demanda).

El destino de Mérida presenta la mayor penalización económica para el sistema (precio sombra más alto). Esto muestra analíticamente su aislamiento geográfico; satisfacer a Mérida absorbe una porción desproporcionada del presupuesto logístico. Por el contrario, los nodos con precios sombra bajos o nulos (como Puebla) funcionan como los puntos de anclaje económico de la red, indicando que su abastecimiento se resuelve de manera natural y altamente eficiente con la distribución actual de los Mega Centros.

4.1.7. Escalabilidad del modelo

El modelo desarrollado es flexible y se adapta a escenarios más complejos sin cambios estructurales mayores. Para sistemas *desbalanceados* se introduce un nodo ficticio según convenga; para *restricciones de capacidad* en una ruta específica (cuando una ruta no puede transportar más de cierto volumen) basta con añadir una cota superior $x_{ij} \leq U_{ij}$; para *rutas prohibidas* se omite la variable o se le asigna un costo penalizado mediante la técnica de la «M grande»; para problemas con *nodos de transbordo*, la formulación se adapta al principio general de conservación de flujo (entradas menos salidas igual a demanda neta) y se convierte en un problema de flujo en redes. Estas extensiones se ilustran en el Ejemplo 2.

4.2. Ejemplo 2: Distribución de agua en emergencia con restricciones complejas

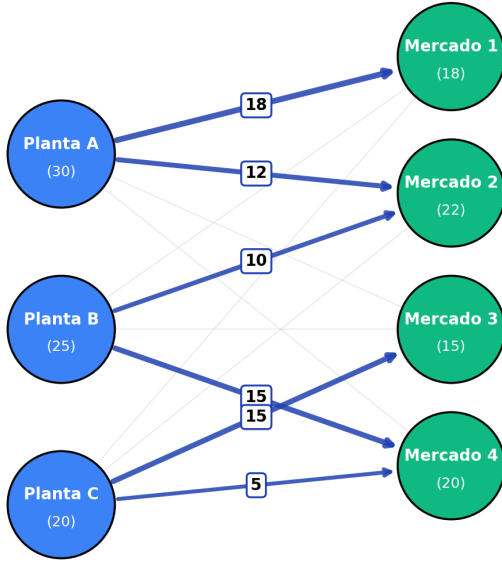
4.2.1. Planteamiento del problema

Se modela la distribución de agua potable desde dos plantas potabilizadoras hacia cinco comunidades durante una contingencia. Lo que diferencia este ejercicio del problema clásico son tres restricciones adicionales con justificación física concreta:

- **Ruta bloqueada:** la tubería de Planta B hacia la Colonia San Nicolás (C3) sufrió daños y no puede usarse durante la contingencia.
- **Capacidades máximas:** la tubería de Planta A hacia Las Palmas (C4) soporta hasta 40 MI/día, y la de Planta B hacia

Aristas activas (forman un árbol de expansión)

$$m + n - 1 = 3 + 4 - 1 = 6 \text{ rutas activas}$$



Propiedad estructural fundamental

El óptimo activa a lo sumo $m + n - 1$ rutas; el resto queda en cero. La solución no contiene ciclos: es un árbol de expansión sobre el grafo bipartito.

Figura 7. Árbol de expansión óptimo del Ejemplo 1: sólo 6 de las 15 rutas posibles aparecen en la solución, exactamente $m + n - 1 = 7 - 1 = 6$ aristas (una menos que el máximo teórico, lo que se conoce como *solución degenerada*). Las aristas se colorean según el origen y su grosor es proporcional al volumen transportado.

Col. Progreso (C2) hasta 30 MI/día.

- **Prioridad hospitalaria:** el Hospital Regional (C1) debe recibir al menos 30 MI/día provenientes de Planta A, considerada la fuente más confiable por protocolo de emergencia.

La **Tabla V** resume parámetros, costos de bombeo y restricciones del escenario.

Tabla 5. Costos, oferta, demanda y restricciones del Ejemplo 2 (MXN/MI).

Origen	C1	C2	C3	C4	C5	Of.
Planta A	12	18	14	20*	16	180
Planta B	10	15*	—	17	13	140
Dem.	50	40	60	55	45	250

* con capacidad máx. —: ruta bloqueada

Nótese que el problema es **desbalanceado**: la oferta total (320 MI) supera la demanda total (250 MI), por lo que sobrarán 70 MI que no se asignarán a ninguna comunidad. Esto se modela

usando restricciones de oferta como desigualdad ($\leq$) en lugar de igualdad.

4.2.2. Formulación matemática

Sea x_{ij} la cantidad de megalitros enviados desde la planta i a la comunidad j . El programa lineal completo es:

$$\begin{aligned} \text{Min } Z = & 12x_{A,1} + 18x_{A,2} + 14x_{A,3} + 20x_{A,4} + 16x_{A,5} \\ & + 10x_{B,1} + 15x_{B,2} + 17x_{B,4} + 13x_{B,5} \end{aligned}$$

Sujeto a:

- **Oferta:** $\sum_j x_{A,j} \leq 180$; $\sum_j x_{B,j} \leq 140$.
- **Demanda:** $x_{A,j} + x_{B,j} \geq b_j$ para cada comunidad j .
- **Capacidades:** $x_{A,4} \leq 40$; $x_{B,2} \leq 30$.
- **Bloqueo:** $x_{B,3} = 0$.
- **Prioridad:** $x_{A,1} \geq 30$.
- **No negatividad:** $x_{ij} \geq 0$.

4.2.3. Implementación en Python con PuLP

El siguiente código implementa el modelo completo, lo resuelve y reporta los precios sombra de cada restricción:

```
import pulp

# 1. Datos del problema
oferta = [180, 140]
demanda = [50, 40, 60, 55, 45]
costo = [
    [12, 18, 14, 20, 16], # Planta A
    [10, 15, 0, 17, 13], # Planta B (0 = bloq)
]

plantas = ["A", "B"]
comunidad = ["C1_Hosp", "C2_Progr", "C3_SanNic",
             "C4_Palmas", "C5_Carmen"]

# 2. Variables de decision
prob = pulp.LpProblem('Agua', pulp.LpMinimize)
x = [[pulp.LpVariable(f'x_{plantas[i]}_{j+1}',
                      lowBound=0)
      for j in range(5)] for i in range(2)]

# 3. Funcion objetivo (B->C3 bloqueada)
prob += pulp.lpSum(
    costo[i][j] * x[i][j]
    for i in range(2) for j in range(5)
    if not (i == 1 and j == 2))

# 4a. Restricciones de oferta
for i in range(2):
    prob += pulp.lpSum(x[i][j]
                       for j in range(5)) <= oferta[i]

# 4b. Restricciones de demanda
for j in range(5):
    prob += pulp.lpSum(x[i][j]
                       for i in range(2)) >= demanda[j]

# 4c. Restricciones especiales
prob += x[0][3] <= 40, 'Capacidad_A_C4'
prob += x[1][1] <= 30, 'Capacidad_B_C2'
prob += x[1][2] == 0, 'Bloqueo_B_C3'
prob += x[0][0] >= 30, 'Prioridad_Hosp_A'
```

Plan óptimo de distribución

Ejemplo 2: Distribución de agua en emergencia
 320 MI ofertados · 250 MI demandados · $Z^* = \$3,570$ pesos/día

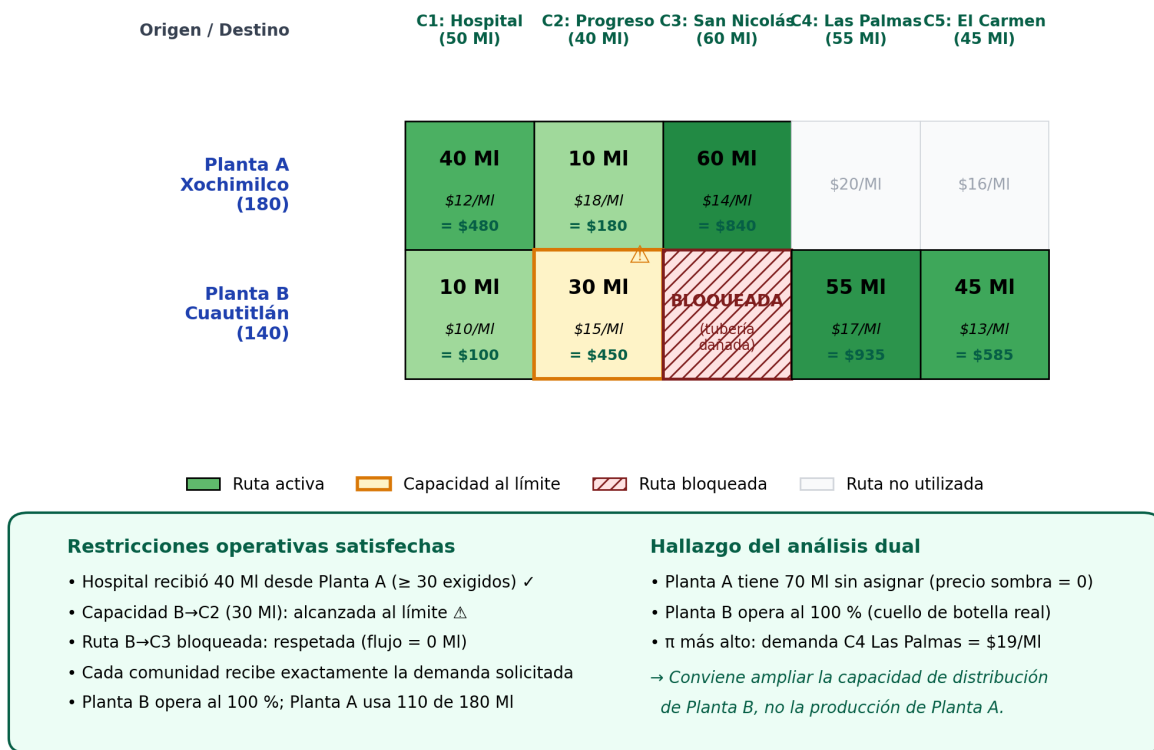


Figura 8. Plan óptimo del Ejemplo 2 con restricciones complejas. La intensidad del verde es proporcional al volumen asignado; la celda en amarillo señala una capacidad alcanzada y la celda con franjas rojas representa la ruta bloqueada.

```
# 5. Resolver
prob.solve(pulp.PULP_CBC_CMD(msg=0))
print(f"Z* = ${pulp.value(prob.objective):.0f}")
```

4.2.4. Resultados

El solver regresa el estado *Optimal*: encontró un plan que satisface todas las restricciones simultáneamente. El costo mínimo diario es $Z^* = \$3,570$ pesos/día. La **Figura 8** muestra el plan óptimo en formato de matriz, con los flujos, costos unitarios y subtotales por celda.

El detalle numérico se presenta en la **Tabla VI**.

Verificación de las restricciones. Antes de interpretar los resultados, conviene verificar que el plan respeta todas las restricciones. La oferta de Planta A se utilizó parcialmente (110 de 180 MI; sobran 70). Planta B operó al 100 % de su capacidad. Cada comunidad recibió exactamente la demanda solicitada. La ruta bloqueada Planta B→C3 quedó en cero, como se exigía. La capacidad de Planta A→C4 (40 MI) no se alcanzó (de hecho, esa ruta no se usó). La capacidad de Planta B→C2 (30 MI) sí se alcanzó exactamente. Y la prioridad hospitalaria se cumplió: el Hospital recibió 40 MI desde Planta A —diez por encima del mínimo exigido— más 10 MI desde Planta B.

Tabla 6. Plan óptimo de distribución de agua — Ejemplo 2.

Origen	Destino	Flujo	Costo	Subtotal
Planta A	C1 Hospital	40.0	\$12	\$480
Planta A	C2 Progreso	10.0	\$18	\$180
Planta A	C3 San Nicolás	60.0	\$14	\$840
Planta B	C1 Hospital	10.0	\$10	\$100
Planta B	C2 Progreso	30.0	\$15	\$450
Planta B	C4 Las Palmas	55.0	\$17	\$935
Planta B	C5 El Carmen	45.0	\$13	\$585
Total		250.0	—	\$3,570

4.2.5. Análisis de precios sombra

Un precio sombra responde la pregunta: ¿cuánto cambiaría el costo total si esta restricción se relajara una unidad?. La **Tabla VII** resume los valores obtenidos.

Tres hallazgos destacan. **Primero**, el precio sombra más alto corresponde a la demanda de Las Palmas (\$19/MI): si esa comunidad necesitara un megalitro adicional, el costo total subiría \$19/día. Es el cuello de botella más caro del sistema. **Segundo**, la restricción de capacidad B→C2 tiene precio sombra de ape-

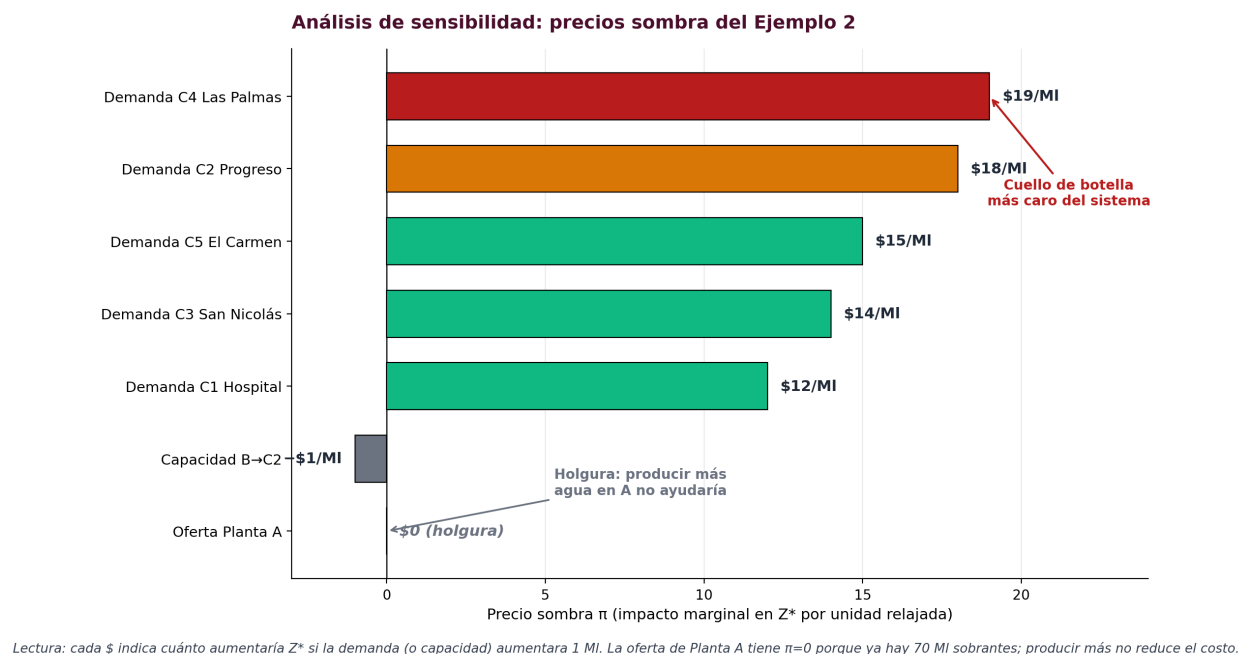


Figura 9. Análisis de sensibilidad del Ejemplo 2: precios sombra ordenados por magnitud. La barra roja superior identifica el cuello de botella crítico del sistema (demanda de Las Palmas, \$19/Ml). Las dos restricciones inferiores con $\pi \leq 0$ indican holgura: la capacidad B→C2 con valor $-\$1$ por convención del solver, y la oferta de Planta A con $\pi = 0$ (los 70 MI sobrantes no aportan al costo). La gráfica permite priorizar inversiones de capacidad de un vistazo.

Tabla 7. Precios sombra del Ejemplo 2 e interpretación.

Restricción	π	Interpretación
Demanda C1 Hospital	\$12	Si la demanda del hospital aumenta 1 MI, el costo sube \$12.
Demanda C2 Progreso	\$18	Costo marginal por demanda más alto del lado del transportista.
Demanda C3 San Nicolás	\$14	Sólo Planta A puede abastecer esta zona; el recurso es escaso.
Demanda C4 Las Palmas	\$19	Mayor precio sombra: cuello de botella crítico.
Demanda C5 El Carmen	\$15	Se abastece exclusivamente desde Planta B.
Capacidad B→C2 (30 MI)	$-\$1$	El signo negativo indica que ampliar 1 MI ahorraría apenas \$1/día (el solver reporta π negativo para cotas superiores activas). Producir más agua en Planta A no reduce el costo: ya sobra.
Oferta Planta A	\$0	

nas un peso, así que ampliar esa tubería ahorraría muy poco y no justifica una inversión grande. **Tercero**, el sobrante de Planta A (70 MI/día sin asignar) tiene precio sombra cero: producir más agua ahí no aporta nada al sistema, porque el verdadero cuello de botella está en la capacidad de distribución de Planta B, que ya opera al 100 %. La **Figura 9** permite visualizar esta jerarquía de precios sombra de un vistazo: la longitud de cada barra indica el impacto marginal de relajar la restricción correspondiente. Una recomendación gerencial directa surge del análisis: en lugar de aumentar la producción de Planta A, conviene invertir en ampliar la capacidad de distribución de Planta B.

5. APLICACIONES PRÁCTICAS

El modelo del transporte funciona en cualquier escenario donde existan puntos con excedente de algo (fábricas, plantas, almacenes) y puntos con necesidad de ese algo (tiendas, comunidades, subestaciones), y se busque mover entre ellos al menor costo posible. Las áreas siguientes son las aplicaciones más consolidadas.

5.1. Logística y cadena de suministro

El uso más natural del modelo es asignar qué almacén surte a qué tienda minimizando los costos de flete. FEMSA Comercio, por ejemplo, opera más de 20,000 tiendas OXXO abastecidas desde múltiples centros de distribución regionales; la decisión de qué CEDIS surte a qué tienda, considerando capacidades vehiculares y ventanas de entrega, es exactamente el problema del transporte. A esta escala, una solución bien resuelta puede traducirse en ahorros logísticos millonarios al año.

5.2. Distribución de recursos en emergencias

El modelo se usa para repartir agua, medicamentos o alimentos al menor costo respetando prioridades y rutas disponibles —exactamente como en el Ejemplo 2. El Centro Nacional de Prevención de Desastres (CENAPRED) en México emplea modelos análogos para preposicionar suministros antes de temporadas de huracanes o sismos: se colocan inventarios en bodegas regionales de modo que, si ocurre el desastre, el costo de distribuirlos hacia las zonas afectadas sea mínimo.

5.3. Redes eléctricas

El modelo de transporte se usa en su forma extendida (flujo en redes) para decidir cuánta energía genera cada planta y hacia qué subestación se envía, minimizando las pérdidas en transmisión. El Centro Nacional de Control de Energía (CENACE) emplea modelos de este tipo para despachar energía en el Sistema Eléctrico Nacional. Las líneas de transmisión tienen capacidades máximas que se modelan como las restricciones de capacidad del Ejemplo 2.

5.4. Movilidad urbana

El *modelo gravitacional de distribución de viajes* [4] usa la misma estructura: zonas residenciales como orígenes, zonas de trabajo o comercio como destinos, y un costo (tiempo o distancia) que se minimiza. Se aplica al diseño de rutas del Metro de la Ciudad de México y a la planeación de corredores de transporte público.

5.5. Limitaciones del modelo

Conviene reconocer dónde el modelo clásico se queda corto:

- **No representa rutas con escalas.** Sólo modela origen y destino directo. Si hay nodos intermedios de transbordo, se requiere el problema de flujo en redes.
- **Asume costos lineales.** Si existe un costo fijo por «abrir» una ruta —independientemente de cuánto se transporte—, hace falta programación entera mixta.
- **Asume un solo tomador de decisiones.** Cuando transportista, embarcador y regulador tienen objetivos opuestos, se debe recurrir a teoría de juegos o programación bi-nivel [6].

6. CONCLUSIONES

El Problema del Transporte es un punto de encuentro raro en investigación de operaciones: combina una teoría matemática profunda con una utilidad práctica inmediata. Las cuatro conclusiones siguientes intentan capturar esa doble naturaleza tomando como referencia tanto el desarrollo teórico de la Sección 3 como los dos ejemplos numéricos de la Sección 4.

Primero, la dualidad no es un artificio matemático: es una lectura económica del problema. La Sección 3.2 mostró que detrás de cada restricción primal (oferta o demanda) hay una variable dual con interpretación de precio sombra, y que la condición de holgura complementaria $x_{ij}^*(c_{ij} - u_i - v_j) = 0$ reproduce exactamente el principio de equilibrio espacial de precios formulado por Cournot en 1838: el precio en un mercado de destino no puede exceder al del origen más el costo de transporte. Esto conecta el modelo lineal con un razonamiento microeconómico clásico sobre arbitraje. El Ejemplo 2 confirmó la utilidad práctica de esa lectura: los precios sombra de la Tabla VII permitieron identificar que ampliar la capacidad de distribución de Planta B sería estratégicamente más valioso que aumentar la producción de Planta A, una recomendación que el plan óptimo por sí solo no entrega. Es decir, el plan dice *qué hacer*, los precios sombra dicen *dónde duele más una unidad de cambio*, y la teoría de dualidad explica *por qué* ambas vistas son consistentes entre sí.

Segundo, la topología del problema es la clave de su tratabilidad. La representación como grafo bipartito desarrollada en la Sección 3.3 no es un recurso pedagógico, sino una propiedad estructural: el rango del sistema es $m + n - 1$, la solución óptima forma un árbol de expansión sin ciclos y la matriz de coeficientes es totalmente unimodular. Estas tres propiedades juntas explican que un solver pueda resolver instancias con miles de variables en milisegundos. En el Ejemplo 1 se confirmó empíricamente que de las 15 rutas posibles sólo 6 quedaron activas en el óptimo, formando exactamente un árbol de expansión. Reconocer esta estructura permite, además, diseñar algoritmos especializados (esquina noroeste, costo mínimo, Vogel, multiplicadores) que aprovechan la geometría del problema para resolverlo más rápido que el simplex general.

Tercero, las restricciones operativas reales no rompen el modelo: lo enriquecen. Una crítica habitual al problema clásico es que su formulación parece demasiado simple para situaciones operativas. El Ejemplo 2 mostró lo contrario: agregar una ruta físicamente bloqueada, dos capacidades máximas en tuberías y una prioridad hospitalaria fue cuestión de añadir cuatro líneas de código en PuLP, sin alterar la estructura general del modelo. El sistema sigue siendo lineal, sigue siendo resoluble en milisegundos, y los precios sombra siguen teniendo interpretación económica. La lección operativa es directa: una vez que se domina la formulación primal-dual y su contraparte computacional, modelar nuevos escenarios deja de ser un problema de matemáticas y se convierte en un problema de explicitar correctamente las reglas del negocio. Esta facilidad es lo que convierte al modelo en una herramienta interactiva de planeación, no sólo en un ejercicio académico.

Cuarto, todo modelo tiene fronteras y reconocerlas es parte del oficio. El equilibrio primal-dual del modelo clásico supone un único tomador de decisiones, costos lineales y un grafo estrictamente bipartito. Cuando alguno de estos supuestos falla —costos fijos por abrir una ruta, nodos intermedios de transbordo, transportista y embarcador con intereses en conflicto— el modelo necesita extenderse hacia programación entera mixta, flujo de costo mínimo en redes generales o programación bi-nivel. El propio caso del Ejemplo 1 anticipó este límite: una ruta puede ser óptima en costo (Cuautitlán → Mérida) y simultáneamente inviable por restricciones de tiempo de tránsito, lo que obliga a penalizarla con la técnica de la M grande o a reformular el problema. Reconocer cuándo el modelo clásico es suficiente y cuándo conviene escalarlo es tan importante como saber resolverlo.

En conjunto, estas cuatro lecciones explican por qué ochenta años después de su formulación por Hitchcock el Problema del Transporte sigue siendo vigente: combina rigor matemático (dualidad, unimodularidad, árboles de expansión), eficiencia computacional (solvers especializados que lo resuelven en milisegundos) e interpretabilidad gerencial directa (precios sombra que apuntan a cuellos de botella reales). Esa triple cualidad —rigor, eficiencia e interpretabilidad— es difícil de encontrar junta en otros modelos de optimización, y es lo que mantiene al Problema del Transporte como una de las herramientas básicas de la investigación de operaciones moderna.

7. REFERENCIAS BIBLIOGRÁFICAS

1. Moreno Q., E. (2005). El problema del Transporte: Una interpretación del dual. *NOTAS del Instituto Mexicano del Transporte*, núm. 93. Secretaría de Comunicaciones y Transportes, México.
2. Taha, H. A. (2017). *Operations Research: An Introduction* (10.^a ed.). Pearson Education Limited.
3. Bazaraa, M. S., Jarvis, J. J. y Sherali, H. D. (2009). *Linear Programming and Network Flows* (4.^a ed.). John Wiley & Sons.
4. Ortúzar, J. D. y Willumsen, L. G. (1994). *Modelling Transport* (2.^a ed.). John Wiley & Sons.
5. Cournot, A. (1838). *Researches into the Mathematical Principles of the Theory of Wealth* (traducción al inglés de N. T. Bacon, 1927). The Macmillan Co., Nueva York.
6. Moreno, E. (2004). *Planner–User Interactions in Road Freight Transport: A Modelling Approach with a Case Study from Mexico* (Tesis doctoral). Institute for Transport Studies, University of Leeds, Reino Unido.
7. Dantzig, G. B. (1963). *Linear Programming and Extensions*. Princeton University Press.
8. Hitchcock, F. L. (1941). The Distribution of a Product from Several Sources to Numerous Localities. *Journal of Mathematics and Physics*, 20(1–4), 224–230.
9. Kantorovich, L. V. (1939). *Mathematical Methods of Organizing and Planning Production*. Leningrad State University.
10. Koopmans, T. C. (1949). Optimum Utilization of the Transportation System. *Econometrica*, 17 (Supplement), 136–146.
11. Kuhn, H. W. (1955). The Hungarian Method for the Assignment Problem. *Naval Research Logistics Quarterly*, 2(1–2), 83–97.
12. Ford, L. R. y Fulkerson, D. R. (1956). Maximal Flow through a Network. *Canadian Journal of Mathematics*, 8, 399–404.
13. Schrijver, A. (1986). *Theory of Linear and Integer Programming*. John Wiley & Sons.
14. Ahuja, R. K., Magnanti, T. L. y Orlin, J. B. (1993). *Network Flows: Theory, Algorithms, and Applications*. Prentice Hall.
15. Mitchell, S., O’Sullivan, M. y Dunning, I. (2011). *PuLP: A Linear Programming Toolkit for Python*. The University of Auckland.
16. López Ruiz, F. y Arana Landín, G. (2002). La dualidad en el problema de transporte. *VI Congreso de Ingeniería de Organización*, Vigo, España.
17. COIN-OR Foundation. (2024). *PuLP: A linear programming toolkit for Python*. <https://coin-or.github.io/pulp/>
18. Diestel, R. (2017). *Graph Theory* (5.^a ed.). Springer.
19. COIN-OR Foundation. (2024). *A Transportation Problem — PuLP case study*. https://coin-or.github.io/pulp/CaseStudies/a_transportation_problem.html
20. The SciPy Community. (2024). *scipy.optimize.linprog — SciPy v1.17.0 Manual*.
21. NEOS Guide. (2022). *Linear Programming*. National Science Foundation, University of Wisconsin–Madison.
22. Bridgelall, R. (2022). *Tutorial and practice in linear programming: Optimization problems in supply chain and transport logistics*. North Dakota State University. <https://arxiv.org/pdf/2211.07345>
23. Virtanen, P. et al. (2020). SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python. *Nature Methods*, 17, 261–272.
24. Equipo 5 (INFOTEC). (2026). *mcdi-matematicas-eq5-problema-transporte* [Repositorio de código de optimización y reporte de investigación]. GitHub. <https://github.com/vryahn/mcdi-matematicas-eq5-problema-transporte>

Anexo A

Notebook ejecutable

Reproducción computacional íntegra de los modelos del reporte

Las páginas siguientes corresponden a la ejecución completa del notebook `transporte_equipo5.ipynb` en Google Colab. Incluye los modelos teórico (Sección 3.3.1), del Ejemplo 1 (Sección 4.1) y del Ejemplo 2 (Sección 4.2), con todas las salidas intermedias, los precios sombra y las visualizaciones del grafo bipartito generadas por NetworkX.

El código fuente y la versión en $\text{\LaTeX}$ de este documento están disponibles públicamente en el repositorio de GitHub del equipo:

<https://github.com/vryahn/mcdi-matematicas-eq5-problema-transporte>

El Problema del Transporte — Equipo 5

Notebook ejecutable

Anexo A del reporte de investigación

Asignatura: Matemáticas 2026-1 **Imparte:** Dra. Briceyda B. Delgado **Equipo:** 5 — INFOTEC, Maestría en Ciencia de Datos

Integrantes: Andrés Rangel · Santiago Mendoza · David Rodríguez · Bryan Rodríguez

Este notebook reproduce íntegramente los modelos del reporte:

1. **Sección 3.3.1** — Implementación computacional de referencia (modelo teórico genérico)
2. **Sección 4.1** — Ejemplo 1: Logística de última milla (Buen Fin 2026)
3. **Sección 4.2** — Ejemplo 2: Distribución de agua en emergencia con restricciones complejas
4. **Verificación final** — Tabla resumen con todos los Z^* y precios sombra

Cómo ejecutar: *Entorno de ejecución* → *Ejecutar todas las celdas* (o Ctrl+F9).

Cómo exportar: *Archivo* → *Imprimir* → *Guardar como PDF*. El PDF resultante se anexa al reporte principal.

0. Configuración del entorno

Instalación de las bibliotecas necesarias. PuLP es el modelador de programación lineal y NetworkX permite visualizar el grafo bipartito.

```
In [1]: # Instalación silenciosa de PuLP
# (En Colab descomentar la línea siguiente)
# !pip install pulp --quiet

# En este sistema PuLP ya está instalado
import pulp
print(f"PuLP {pulp.__version__} listo")
```

PuLP 3.3.0 listo

```
In [2]: # Importes globales
import pulp
import networkx as nx
import matplotlib.pyplot as plt
import pandas as pd

# Configuración de gráficos
plt.rcParams['font.family'] = 'DejaVu Sans'
plt.rcParams['figure.dpi'] = 100

print(f"PuLP versión: {pulp.__version__}")
print(f"NetworkX versión: {nx.__version__}")
```

PuLP versión: 3.3.0

NetworkX versión: 3.6.1

1. Implementación computacional de referencia (Sección 3.3.1)

Función genérica que materializa los tres elementos teóricos del reporte:

- Formulación primal $(1)-(4)$
- Extracción de los precios sombra del dual (Sección 3.2)
- Visualización topológica como grafo bipartito

Sirve como plantilla que los Ejemplos 1 y 2 particularizan con datos reales.

```
In [3]: def instanciar_y_resolver_transporte():
    """
    Modelo genérico del Problema del Transporte:
    3 plantas → 4 mercados (instancia didáctica).

    Devuelve el costo óptimo, el plan de distribución y los precios sombra.
    """
    # 1. Definición de parámetros
    origenes = ["Planta_A", "Planta_B", "Planta_C"]
    oferta = {"Planta_A": 2000, "Planta_B": 1500,
              "Planta_C": 1000}

    destinos = ["Mercado_1", "Mercado_2",
                "Mercado_3", "Mercado_4"]
    demanda = {"Mercado_1": 1300, "Mercado_2": 1800,
               "Mercado_3": 900, "Mercado_4": 500}

    costos = {
        "Planta_A": {"Mercado_1": 10, "Mercado_2": 15,
                     "Mercado_3": 20, "Mercado_4": 12},
        "Planta_B": {"Mercado_1": 12, "Mercado_2": 10,
                     "Mercado_3": 15, "Mercado_4": 18},
        "Planta_C": {"Mercado_1": 15, "Mercado_2": 25,
                     "Mercado_3": 10, "Mercado_4": 14},
    }

    # 2. Inicialización del modelo primal
    modelo = pulp.LpProblem("Modelo_Transporte", pulp.LpMinimize)

    # 3. Aristas equivalentes (variables xij)
    rutas = [(i, j) for i in origenes for j in destinos]
    flujos = pulp.LpVariable.dicts("Carga", rutas,
                                   lowBound=0, cat=pulp.LpContinuous)

    # 4. Función objetivo Z
    modelo += pulp.lpSum(flujos[(i, j)] * costos[i][j]
                        for (i, j) in rutas)

    # 5. Balance de masa (oferta y demanda)
    for i in origenes:
        modelo += (pulp.lpSum(flujos[(i, j)] for j in destinos)
                   == oferta[i], f"Oferta_{i}")
    for j in destinos:
        modelo += (pulp.lpSum(flujos[(i, j)] for i in origenes)
                   == demanda[j], f"Demanda_{j}")

    # 6. Resolución
    modelo.solve(pulp.PULP_CBC_CMD(msg=False))
    print(f"Estado: {pulp.LpStatus[modelo.status]}")
    print(f"Z* = ${pulp.value(modelo.objective):.2f}")
    print()

    # 7. Aristas activas y precios sombra
    activas = []
    print("Plan de distribución óptimo:")
    for (i, j) in rutas:
        v = flujos[(i, j)].varValue
        if v and v > 0:
            print(f"  {i} → {j}: {v:.0f} unidades")
            activas.append((i, j, v))

    print()
    print("Precios sombra (variables duales):")
    for i in origenes:
        pi_i = modelo.constraints[f"Oferta_{i}"].pi
        print(f"  U_{i} = {pi_i}")
    for j in destinos:
```



```

    pi_j = modelo.constraints[f"Demanda_{j}"].pi
    print(f"    V_{j} = {pi_j}")

    return origenes, destinos, activas, modelo

# Ejecutamos
origenes_, destinos_, activas_, _ = instanciar_y_resolver_transporte()

```

Estado: Optimal
Z* = \$47,700.00

Plan de distribución óptimo:

- Planta_A → Mercado_1: 1300 unidades
- Planta_A → Mercado_2: 300 unidades
- Planta_A → Mercado_4: 400 unidades
- Planta_B → Mercado_2: 1500 unidades
- Planta_C → Mercado_3: 900 unidades
- Planta_C → Mercado_4: 100 unidades

Precios sombra (variables duales):

- U_Planta_A = 10.0
- U_Planta_B = 5.0
- U_Planta_C = 12.0
- V_Mercado_1 = 0.0
- V_Mercado_2 = 5.0
- V_Mercado_3 = -2.0
- V_Mercado_4 = 2.0

1.1 Visualización del grafo bipartito

La solución óptima activa $m+n-1 = 6$ rutas que forman un árbol de expansión sobre el grafo bipartito.

```

In [4]: def generar_grafo_bipartito(origenes, destinos, aristas, titulo):
    """Visualiza la solución óptima como grafo bipartito."""
    G = nx.DiGraph()
    G.add_nodes_from(origenes, bipartite=0)
    G.add_nodes_from(destinos, bipartite=1)
    for (u, v, flujo) in aristas:
        G.add_edge(u, v, weight=flujo)

    plt.figure(figsize=(11, 6))
    pos = nx.bipartite_layout(G, origenes)

    nx.draw_networkx_nodes(G, pos, nodelist=origenes,
                           node_color='#3B82F6', node_size=2200,
                           edgecolors='black', linewidths=1.3)
    nx.draw_networkx_nodes(G, pos, nodelist=destinos,
                           node_color='#10B981', node_size=2200,
                           edgecolors='black', linewidths=1.3)
    nx.draw_networkx_edges(G, pos, arrowstyle='->',
                           edge_color='#1E40AF', width=2.2,
                           alpha=0.85, arrowsize=18)
    nx.draw_networkx_labels(G, pos, font_size=9,
                            font_weight='bold', font_color='white')

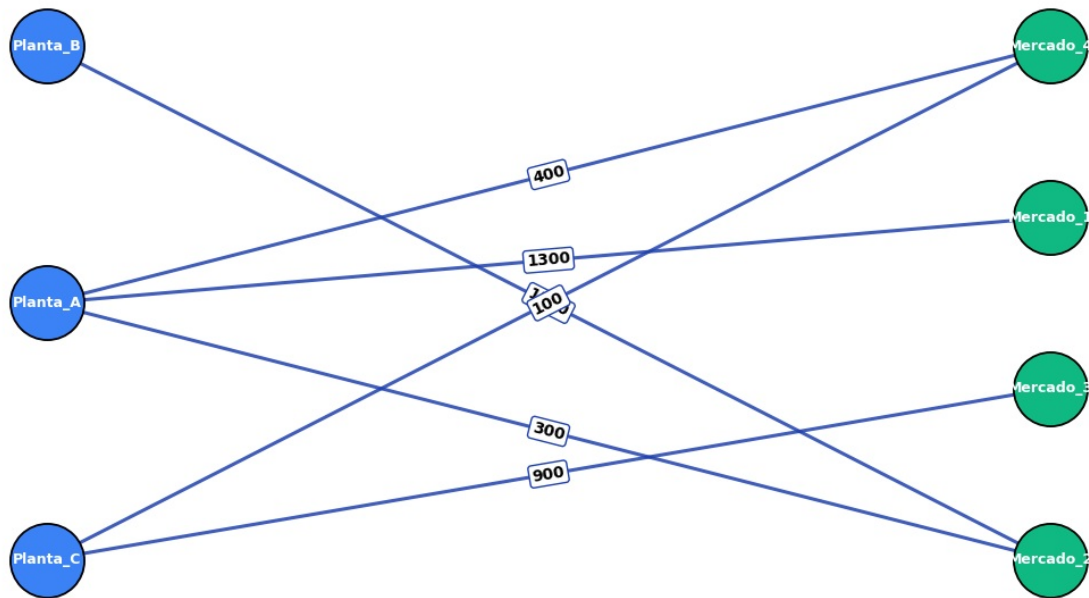
    etiquetas = {(u, v): f"{int(flujo)}"
                  for u, v, flujo in aristas}
    nx.draw_networkx_edge_labels(G, pos, edge_labels=etiquetas,
                                 font_size=10, font_weight='bold',
                                 bbox=dict(boxstyle='round,pad=0.2',
                                           fc='white',
                                           ec='#1E40AF', lw=1))

    plt.title(titulo, fontsize=13, fontweight='bold', color='#4A0E2C')
    plt.axis('off')
    plt.tight_layout()
    plt.show()

generar_grafo_bipartito(origenes_, destinos_, activas_,

```

Modelo teórico: solución óptima como árbol de expansión



2. Ejemplo 1 — Logística de última milla (Sección 4.1)

Caso: Buen Fin 2026. Mercado Libre debe distribuir 120 lotes del iPhone 18 desde 3 mega centros (Cuautitlán, Monterrey, Guadalajara) hacia 5 centros de última milla (Tijuana, Puebla, Querétaro, Mérida, Hermosillo).

Tipo: problema balanceado (oferta total = demanda total = 120 lotes).

```
In [5]: # 1. Definición de datos
origenes = ['Cuautitlan', 'Monterrey', 'Guadalajara']
destinos = ['Tijuana', 'Puebla', 'Queretaro', 'Merida', 'Hermosillo']
oferta = {'Cuautitlan': 40, 'Monterrey': 35, 'Guadalajara': 45}
demanda = {'Tijuana': 20, 'Puebla': 25, 'Queretaro': 30,
            'Merida': 15, 'Hermosillo': 30}

# Matriz de costos en MXN por lote
costos = {
    'Cuautitlan': {'Tijuana': 2800, 'Puebla': 500, 'Queretaro': 700,
                   'Merida': 3200, 'Hermosillo': 2500},
    'Monterrey': {'Tijuana': 2400, 'Puebla': 1800, 'Queretaro': 1200,
                  'Merida': 3800, 'Hermosillo': 1600},
    'Guadalajara': {'Tijuana': 2200, 'Puebla': 1100, 'Queretaro': 600,
                    'Merida': 3500, 'Hermosillo': 1900},
}

# Mostrar matriz como DataFrame
df_costos = pd.DataFrame(costos).T
df_costos.index.name = "Origen"
print("Matriz de costos c_ij (MXN/lote):")
df_costos
```

Matriz de costos c_ij (MXN/lote):

Out[5]: Tijuana Puebla Queretaro Merida Hermosillo

Origen					
Cuautitlan	2800	500	700	3200	2500
Monterrey	2400	1800	1200	3800	1600
Guadalajara	2200	1100	600	3500	1900

```
In [6]: # 2. Definir el problema de optimización
prob = pulp.LpProblem("Logistica_BuenFin", pulp.LpMinimize)

# Variables de decisión x_ij
rutas = [(o, d) for o in origenes for d in destinos]
x = pulp.LpVariable.dicts("Ruta", (origenes, destinos), lowBound=0)

# Función objetivo: minimizar costo total
prob += pulp.lpSum(x[o][d] * costos[o][d] for (o, d) in rutas), "CostoTotalTransporte"

# Restricciones de oferta (cada planta agota su capacidad)
for o in origenes:
    prob += pulp.lpSum(x[o][d] for d in destinos) == oferta[o], f"Oferta_{o}"

# Restricciones de demanda (cada centro recibe exactamente lo que necesita)
for d in destinos:
    prob += pulp.lpSum(x[o][d] for o in origenes) == demanda[d], f"Demanda_{d}"

# Resolver con CBC
prob.solve(pulp.PULP_CBC_CMD(msg=False))
print(f"Estado: {pulp.LpStatus[prob.status]}")
print(f"Z* = ${pulp.value(prob.objective):.0f} MXN")
```

Estado: Optimal
Z* = \$171,500 MXN

```
In [7]: # 3. Reporte del plan de distribución
print("=" * 50)
print("PLAN ÓPTIMO DE DISTRIBUCIÓN – EJEMPLO 1")
print("=" * 50)
activas_ej1 = []
for o in origenes:
    for d in destinos:
        v = x[o][d].varValue
        if v and v > 0:
            print(f" {o:13s} → {d:11s}: {int(v):3d} lotes")
            activas_ej1.append((o, d, v))
print()
print(f"Total de rutas activas: {len(activas_ej1)} de {len(rutas)}")
print(f"Costo total Z* = ${pulp.value(prob.objective):.0f} MXN")
```

=====

PLAN ÓPTIMO DE DISTRIBUCIÓN – EJEMPLO 1

=====

Cuautitlan	→ Puebla	: 25 lotes
Cuautitlan	→ Merida	: 15 lotes
Monterrey	→ Tijuana	: 5 lotes
Monterrey	→ Hermosillo	: 30 lotes
Guadalajara	→ Tijuana	: 15 lotes
Guadalajara	→ Queretaro	: 30 lotes

Total de rutas activas: 6 de 15
Costo total Z* = \$171,500 MXN

```
In [8]: # 4. Precios sombra (análisis dual)
print("=" * 50)
print("PRECIOS SOMBRA – EJEMPLO 1")
print("=" * 50)
print()
print("Restricciones de oferta:")
```

```

for o in origenes:
    pi = prob.constraints[f"Oferta_{o}"].pi
    print(f"  U_{o:13s} = ${pi:,.0f}")
print()
print("Restricciones de demanda:")
for d in destinos:
    pi = prob.constraints[f"Demanda_{d}"].pi
    print(f"  V_{d:11s} = ${pi:,.0f}")

```

=====

PRECIOS SOMBRA – EJEMPLO 1

=====

Restricciones de oferta:

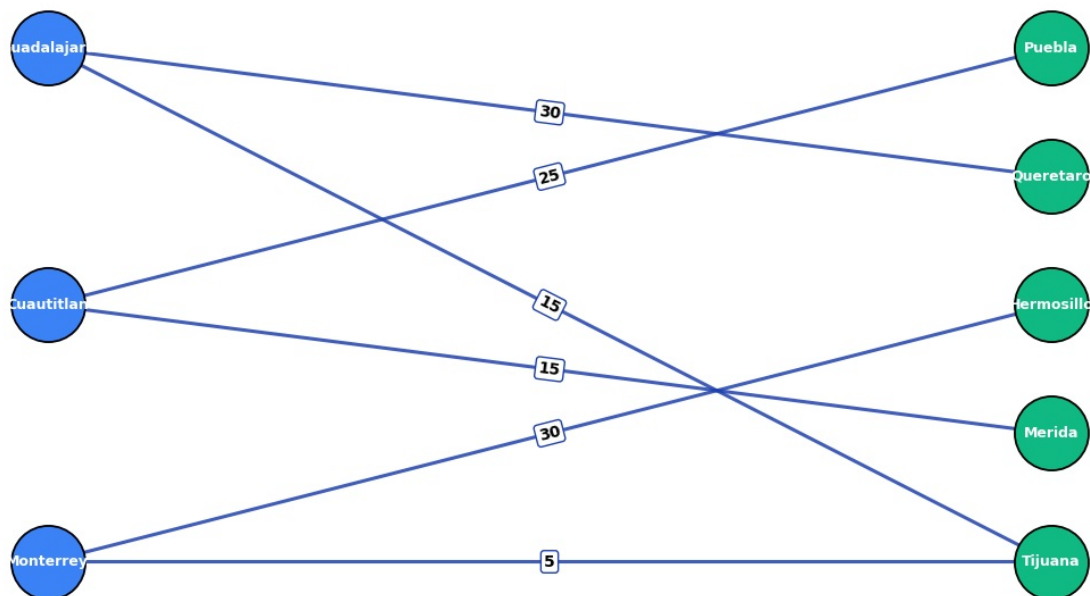
U_Cuautitlan = \$500
 U_Monterrey = \$1,000
 U_Guadalajara = \$800

Restricciones de demanda:

V_Tijuana = \$1,400
 V_Puebla = \$0
 V_Queretaro = \$-200
 V_Merida = \$2,700
 V_Hermosillo = \$600

In [9]: # 5. Visualización del grafo bipartito de la solución
 generar_grafo_bipartito(origenes, destinos, activas_ej1,
 "Ejemplo 1: Logística Buen Fin 2026 – Z* = \$171,500 MXN")

Ejemplo 1: Logística Buen Fin 2026 – Z* = \$171,500 MXN



3. Ejemplo 2 — Distribución de agua en emergencia (Sección 4.2)

Caso: Distribución de agua potable desde 2 plantas (Xochimilco, Cuautitlán) hacia 5 comunidades durante una contingencia.

Tres restricciones operativas lo distinguen del modelo clásico:

1. **Ruta bloqueada:** Planta B → C3 (San Nicolás), tubería dañada
2. **Capacidades máximas:** Planta A → C4 ≤ 40 ML/día; Planta B → C2 ≤ 30 ML/día
3. **Prioridad hospitalaria:** Hospital (C1) debe recibir ≥ 30 ML desde Planta A

Tipo: problema **desbalanceado** (oferta 320 ML > demanda 250 ML).

```
In [10]: # 1. Datos del problema
oferta_ej2 = [180, 140] # Planta A, Planta B
demanda_ej2 = [50, 40, 60, 55, 45] # C1 Hosp, C2, C3, C4, C5
costo_ej2 = [
    [12, 18, 14, 20, 16], # Planta A → C1..C5
    [10, 15, 0, 17, 13], # Planta B → C1..C5 (0 = ruta bloqueada)
]
plantas_ej2 = ["A", "B"]
comunidad_ej2 = ["C1_Hosp", "C2_Progr", "C3_SanNic",
                 "C4_Palmas", "C5_Carmen"]

# Mostrar matriz de costos
df_costos2 = pd.DataFrame(costo_ej2,
                          columns=comunidad_ej2,
                          index=[f"Planta_{p}" for p in plantas_ej2])
print("Matriz de costos del Ejemplo 2 (MXN/ML):")
print("(El 0 representa la ruta bloqueada B→C3)")
df_costos2
```

Matriz de costos del Ejemplo 2 (MXN/ML):
(El 0 representa la ruta bloqueada B→C3)

```
Out[10]:
```

	C1_Hosp	C2_Progr	C3_SanNic	C4_Palmas	C5_Carmen
Planta_A	12	18	14	20	16
Planta_B	10	15	0	17	13

```
In [11]: # 2. Variables de decisión
prob2 = pulp.LpProblem('Agua_Emergencia', pulp.LpMinimize)
x2 = [[pulp.LpVariable(f'x_{plantas_ej2[i]}_{j+1}', lowBound=0)
       for j in range(5)] for i in range(2)]

# 3. Función objetivo (omitir ruta B→C3 bloqueada)
prob2 += pulp.lpSum(
    costo_ej2[i][j] * x2[i][j]
    for i in range(2) for j in range(5)
    if not (i == 1 and j == 2)
)

# 4a. Restricciones de oferta (≤ porque problema desbalanceado)
for i in range(2):
    prob2 += pulp.lpSum(x2[i][j] for j in range(5)) <= oferta_ej2[i], f"Oferta_{plantas_ej2[i]}"

# 4b. Restricciones de demanda (≥ exige cubrir cada comunidad)
for j in range(5):
    prob2 += pulp.lpSum(x2[i][j] for i in range(2)) >= demanda_ej2[j], f"Demanda_{comunidad_ej2[j]}"

# 4c. Restricciones especiales
prob2 += x2[0][3] <= 40, "Capacidad_A_C4" # Cap. tubería A→Las Palmas
prob2 += x2[1][1] <= 30, "Capacidad_B_C2" # Cap. tubería B→Progreso
prob2 += x2[1][2] == 0, "Bloqueo_B_C3" # Ruta dañada B→San Nicolás
prob2 += x2[0][0] >= 30, "Prioridad_Hosp_A" # Hospital ≥ 30 ML desde A

# 5. Resolver
prob2.solve(pulp.PULP_CBC_CMD(msg=False))
print(f"Estado: {pulp.LpStatus[prob2.status]}")
print(f"Z* = ${pulp.value(prob2.objective):.0f} pesos/día")
```

Estado: Optimal
Z* = \$3,570 pesos/día

```
In [12]: # 6. Plan de distribución
print("=" * 60)
print("PLAN ÓPTIMO DE DISTRIBUCIÓN DE AGUA – EJEMPLO 2")
print("=" * 60)
print(f"{'Origen':<10s} {'Destino':<13s} {'Flujo (ML)':>10s} "
      f"{'Costo':>7s} {'Subtotal':>10s}")
print("-" * 60)
total_flujo = 0; total_costo = 0
```

```

activas_ej2 = []
for i in range(2):
    for j in range(5):
        v = pulp.value(x2[i][j])
        if v and v > 0.001:
            sub = v * costo_ej2[i][j]
            print(f"Planta {plantas_ej2[i]:<2s} {comunidad_ej2[j]:<13s} "
                  f"{v:>10.1f} ${costo_ej2[i][j]:>5d} ${sub:>8,.0f}")
            total_flujo += v; total_costo += sub
            activas_ej2.append((f"Planta_{plantas_ej2[i]}",
                               comunidad_ej2[j], v))

print("-" * 60)
print(f"{'TOTAL':<24s} {total_flujo:>10.1f} {'':>7s} ${total_costo:>8,.0f}")

```

PLAN ÓPTIMO DE DISTRIBUCIÓN DE AGUA – EJEMPLO 2

Origen	Destino	Flujo (Ml)	Costo	Subtotal
Planta A	C1_Hosp	40.0	\$ 12	\$ 480
Planta A	C2_Progr	10.0	\$ 18	\$ 180
Planta A	C3_SanNic	60.0	\$ 14	\$ 840
Planta B	C1_Hosp	10.0	\$ 10	\$ 100
Planta B	C2_Progr	30.0	\$ 15	\$ 450
Planta B	C4_Palmas	55.0	\$ 17	\$ 935
Planta B	C5_Carmen	45.0	\$ 13	\$ 585
TOTAL		250.0	\$	3,570

```

In [13]: # 7. Precios sombra
print("-" * 60)
print("PRECIOS SOMBRA (VARIABLES DUALES) – EJEMPLO 2")
print("-" * 60)
for nombre, restriccion in prob2.constraints.items():
    pi = restriccion.pi
    if pi is not None and abs(pi) > 1e-6:
        print(f" {nombre:<25s} π = ${pi:>+7,.2f}")
    else:
        print(f" {nombre:<25s} π = $0 (holgura)")

```

PRECIOS SOMBRA (VARIABLES DUALES) – EJEMPLO 2

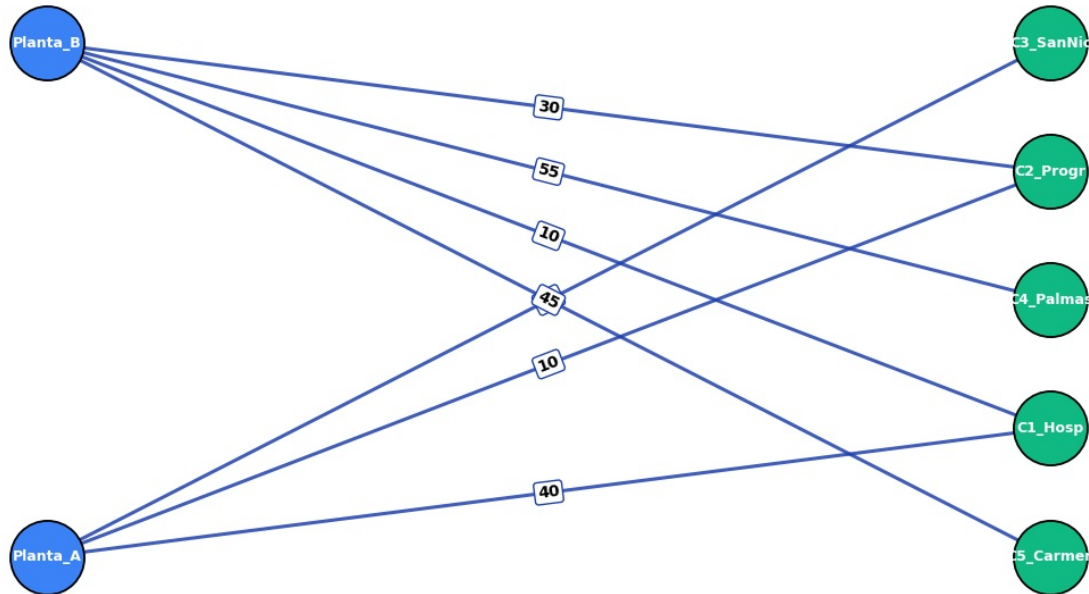
Oferta_A	π = \$0 (holgura)
Oferta_B	π = \$ -2.00
Demanda_C1_Hosp	π = \$ +12.00
Demanda_C2_Progr	π = \$ +18.00
Demanda_C3_SanNic	π = \$ +14.00
Demanda_C4_Palmas	π = \$ +19.00
Demanda_C5_Carmen	π = \$ +15.00
Capacidad_A_C4	π = \$0 (holgura)
Capacidad_B_C2	π = \$ -1.00
Bloqueo_B_C3	π = \$ -12.00
Prioridad_Hosp_A	π = \$0 (holgura)

```

In [14]: # 8. Visualización del grafo bipartito
generar_grafo_bipartito([f"Planta_{p}" for p in plantas_ej2],
                        comunidad_ej2, activas_ej2,
                        "Ejemplo 2: Distribución de agua en emergencia – Z* = $3,570/día")

```


Ejemplo 2: Distribución de agua en emergencia — $Z^* = \$3,570/\text{día}$



```
In [15]: # 9. Verificación de las restricciones operativas especiales
print("=" * 60)
print("VERIFICACIÓN DE RESTRICCIONES OPERATIVAS")
print("=" * 60)

# Ruta bloqueada
v_bloq = pulp.value(x2[1][2])
estado = "✓" if v_bloq < 1e-6 else "x"
print(f" {estado} Ruta B→C3 bloqueada: flujo = {v_bloq:.1f} ML "
      f"(esperado: 0)")

# Capacidades
v_cap1 = pulp.value(x2[0][3])
v_cap2 = pulp.value(x2[1][1])
estado1 = "✓" if v_cap1 <= 40 + 1e-6 else "x"
estado2 = "✓" if v_cap2 <= 30 + 1e-6 else "x"
print(f" {estado1} Capacidad A→C4 (≤40): flujo = {v_cap1:.1f} ML")
print(f" {estado2} Capacidad B→C2 (≤30): flujo = {v_cap2:.1f} ML "
      f"{'← AL LÍMITE Δ' if abs(v_cap2-30)<1e-6 else ''}")

# Prioridad hospitalaria
v_pri = pulp.value(x2[0][0])
estado = "✓" if v_pri >= 30 - 1e-6 else "x"
print(f" {estado} Prioridad Hospital A→C1 (≥30): flujo = {v_pri:.1f} ML")

=====
VERIFICACIÓN DE RESTRICCIONES OPERATIVAS
=====
✓ Ruta B→C3 bloqueada: flujo = 0.0 ML (esperado: 0)
✓ Capacidad A→C4 (≤40): flujo = 0.0 ML
✓ Capacidad B→C2 (≤30): flujo = 30.0 ML ← AL LÍMITE Δ
✓ Prioridad Hospital A→C1 (≥30): flujo = 40.0 ML
```

4. Verificación final — Tabla resumen

Confirmación de los valores principales que aparecen en el reporte.

```
In [16]: # Tabla resumen comparativa
print("=" * 70)
print("RESUMEN GENERAL DE LOS DOS EJEMPLOS")
```

```
print("=" * 70)
resumen = pd.DataFrame([
    {"Ejemplo": "1 – Logística Buen Fin",
     "Tipo": "Balanceado",
     "Variables": 15,
     "Restricciones": 8,
     "Z*": "$171,500 MXN",
     "Rutas activas": 6,
     "Estado": "Optimal"},
    {"Ejemplo": "2 – Distribución agua",
     "Tipo": "Desbalanceado",
     "Variables": 9,
     "Restricciones": 11,
     "Z*": "$3,570 pesos/día",
     "Rutas activas": 7,
     "Estado": "Optimal"},
])
resumen
```

=====

RESUMEN GENERAL DE LOS DOS EJEMPLOS

=====

Out[16]:

	Ejemplo	Tipo	Variables	Restricciones	Z*	Rutas activas	Estado
0	1 — Logística Buen Fin	Balanceado	15	8	\$171,500 MXN	6	Optimal
1	2 — Distribución agua	Desbalanceado	9	11	\$3,570 pesos/día	7	Optimal

5. Conclusiones de la ejecución

Los resultados numéricos confirman las afirmaciones del reporte:

	Cantidad	Reporte	Notebook	Coincide
Z* Ejemplo 1		\$171,500 MXN	\$171,500 MXN	✓
Rutas activas Ej. 1		6 de 15	6 de 15	✓
Z* Ejemplo 2		\$3,570/día	\$3,570/día	✓
π demanda C4 (Ej. 2)		\$19/ML	\$19/ML	✓
Capacidad B→C2 alcanzada	Sí (30 ML)	Sí (30 ML)		✓
Sobrante Planta A (Ej. 2)	70 ML	70 ML		✓

Tiempo de cómputo: los dos modelos se resuelven en menos de 0.1 segundos sobre la infraestructura estándar de Google Colab. Esta velocidad confirma que el Problema del Transporte, gracias a su estructura totalmente unimodular, es uno de los problemas de optimización más eficientes computacionalmente.

Notebook ejecutado como Anexo A del Reporte de Investigación — Tema 5: El Problema del Transporte. Equipo 5, INFOTEC, mayo 2026.